Lab Manual

Programming For AI
AI-212

By

Abdul Haseeb Arif **2023-BSAI-033**

Submitted to:
Sir.Samraiz

Bachelor of Science in Artificial Intelligence
DEPARTMENT OF COMPUTER SCIENCES

Table OF Content

Lab 1

➢ Python

- Python is a high-level, easy-to-learn programming language.
- It uses simple English-like syntax, which makes it beginner-friendly.
- Python is widely used for web development, data analysis, and automation.

➢ Python Programming:

- Python programming allows developers to write fewer lines of code to perform tasks.
- It supports object-oriented, procedural, and functional programming.
- Python has a large collection of libraries that make programming faster and easier.

➢ Python in Programming & AI:

- Python is one of the most popular languages for Artificial Intelligence (AI).
- It is used in machine learning, deep learning, and data science.
- Libraries like NumPy, Pandas, Scikit-learn, and TensorFlow make Python powerful for AI applications.

➢ Basic Python Programming:

➢ Q1. Banking Transaction Validator with PIN:

- **Input:**

```
balance = 70000
correct_pin = "2234"
attempts = 0

while attempts < 3:
    pin = input("Enter 4-digit PIN: ")
    if pin == correct_pin:
        print("Balance:", balance)
        amount = int(input("Enter withdrawal amount: "))
        service_charge = amount * 0.02
        total = amount + service_charge

        if total <= balance:
            balance -= total
            print("Withdrawal successful")
            print("Service Charge:", service_charge)
            print("Remaining Balance:", balance)
        else:
            print("Insufficient balance")
        break
    else:
        attempts += 1
        print("Invalid PIN")

if attempts == 3:
    print("Card Blocked")
```

- **Output:**

```
Enter 4-digit PIN:  2234
Balance: 70000
Enter withdrawal amount:  50000
Withdrawal successful
Service Charge: 1000.0
Remaining Balance: 19000.0
```

➢ Q2. Grocery Store Inventory System:

- **Input:**

```
inventory = {
    "apple": {"stock": 50, "price": 100},
    "banana": {"stock": 100, "price": 50},
    "milk": {"stock": 30, "price": 150}
}

total = 0
item = input("Enter item: ")
qty = int(input("Enter quantity: "))

if item in inventory and inventory[item]["stock"] >= qty:
    inventory[item]["stock"] -= qty
    total = inventory[item]["price"] * qty
else:
    print("Item not available")

if total >= 500:
    discount = 0.20
elif total >= 300:
    discount = 0.15
else:
    discount = 0.10

total -= total * discount
tax = total * 0.05
print("Final Bill:", total + tax)
```

➢ Output:

```
Enter item:  apple
Enter quantity:  2
Final Bill: 189.0
```

Q3. Student Grading with Subject Weightage:

- **Input:**

```
weights = (0.2, 0.2, 0.2, 0.2, 0.2)
marks = []
weighted_sum = 0

for i in range(5):
    m = int(input(f"Enter marks for subject {i+1}: "))
    marks.append(m)
    weighted_sum += m * weights[i]

percentage = weighted_sum

if percentage >= 80:
    grade = "A"
elif percentage >= 65:
    grade = "B"
elif percentage >= 50:
    grade = "C"
elif percentage >= 40:
    grade = "D"
else:
    grade = "Fail"

print("Grade:", grade)
print("Scholarship:", percentage >= 80)
```

- **Output:**

```
Enter marks for subject 1:  99
Enter marks for subject 2:  98
Enter marks for subject 3:  97
Enter marks for subject 4:  95
Enter marks for subject 5:  94
Grade: A
Scholarship: True
```

➢ Q4. BMI + Health Recommendation

- **Input:**

```
weight = float(input("Weight (kg): "))
height = float(input("Height (m): "))
age = int(input("Age: "))

bmi = weight / (height ** 2)
print("BMI:", bmi)

if bmi < 18.5:
    print("Underweight: Eat healthy and exercise lightly")
elif bmi < 25:
    print("Normal: Maintain diet")
elif bmi < 30:
    print("Overweight: Exercise regularly")
else:
    print("Obese: Strict diet and doctor consultation")
```

- **Output:**

```
Weight (kg):  60
Height (m):  5.5
Age:  20
BMI: 1.9834710743801653
Underweight: Eat healthy and exercise lightly
```

➢ Q5. Telecom Prepaid Recharge System:

- **Input:**

```
rates = {"call": 2, "sms": 1, "data": 0.5}
balance = 1000
initial = balance

calls = int(input("Call minutes: "))
sms = int(input("SMS count: "))
data = int(input("Data MB: "))

balance -= (calls * rates["call"] + sms * rates["sms"] + data * rates["data"])

if balance < initial * 0.2:
    print("Recharge Reminder")

recharge = int(input("Recharge amount: "))
if recharge > 1000:
    recharge += 100

balance += recharge
print("Final Balance:", balance)
```

- **Output:**

```
Call minutes:  55
SMS count:  55
Data MB:  6000
Recharge Reminder
Recharge amount:  3000
Final Balance: 935.0
```

➢ Q6. E-Commerce Discount Engine with Membership Levels:

- **Input:**

```
amount = float(input("Bill amount: "))
member = input("Membership (Regular/Gold/Platinum): ")

discount = 0
if member == "Gold":
    discount = 0.10
elif member == "Platinum":
    discount = 0.15

amount -= amount * discount

if amount > 10000:
    amount -= amount * 0.05

tax = amount * 0.05
print("Final Bill:", amount + tax)
```

- **Output:**

```
Bill amount:  55000
Membership (Regular/Gold/Platinum):  platinum
Final Bill: 54862.5
```

➢ Q7. Hospital Emergency Unit with Multi-Condition Triage:

```
temp = float(input("Temperature: "))
pulse = int(input("Pulse: "))
oxygen = int(input("Oxygen: "))
bp = int(input("BP: "))

if oxygen < 85 or bp < 90:
    print("Critical → Admit in ICU")
elif oxygen < 92 or temp > 102:
    print("Urgent → Immediate Doctor")
elif pulse > 100:
    print("Observation")
else:
    print("Stable")
```

- **Output:**

```
Temperature:  37
Pulse:  37
Oxygen:  38
BP:  70
Critical → Admit in ICU
```

➢ **Q8. Online Examination Grader with Negative Marking:**

- **Input:**

```
total = int(input("Total Marks: "))
obtained = float(input("Obtained Marks: "))
wrong = int(input("Wrong Answers: "))

obtained -= wrong * 0.25
percentage = (obtained / total) * 100

print("Percentage:", percentage)

if percentage >= 60:
    print("Division: First")
    print("Scholarship Eligible")
elif percentage >= 45:
    print("Division: Second")
else:
    print("Fail")
    print("Repeat Exam")
```

- **Output:**

```
Percentage: 100.0
Division: First
Scholarship Eligible
```

- **Q9. Movie Ticket Booking with Age & Timing Policy:**
  - **Input:**

```
age = int(input("Age: "))
time = int(input("Showtime (24hr): "))
ticket = input("Ticket Type (Normal/3D/IMAX): ")

price = 300 if ticket == "Normal" else 500 if ticket == "3D" else 700

if age < 12 and time >= 22:
    print("Late-night shows not allowed")
else:
    if age >= 60:
        price *= 0.7
    if 18 <= time <= 21:
        price *= 1.1
    print("Final Ticket Price:", price)
```

  - **Output:**

```
Final Ticket Price: 300
```

- **Q10. Weather-based Outfit + Travel Suggestion**
  - **Input:**

```
temp = int(input("Temperature: "))
weather = input("Weather: ")
event = input("Event Type: ")

if weather == "Rainy":
    print("Carry Umbrella")
if weather == "Cold":
    print("Wear Gloves")

if event == "Business" and temp > 35:
    print("Light formal suit")
else:
    print("Casual outfit recommended")
```

- **Output:**

```
Wear Gloves
Light formal suit
```

➢ Q11. ATM Withdrawal with Note Counting + Receipt

- **Input:**

```
amount = int(input("Withdrawal Amount: "))
balance = 50000

notes = [1000, 500, 100, 50, 10, 1]
print("Notes Breakdown:")


for n in notes:
    print(n, ":", amount // n)
    amount %= n

charge = balance * 0.005
balance -= charge

print("Service Charge:", charge)
print("Remaining Balance:", balance)
```

- **Output:**

```
Withdrawal Amount:  40000
Notes Breakdown:
1000 : 40
500 : 0
100 : 0
50 : 0
10 : 0
1 : 0
Service Charge: 250.0
Remaining Balance: 49750.0
```

➢ Q12. Library Fine Calculator with Membership Levels

- **Input:**

```
days = int(input("Days Late: "))
member = input("Membership (Normal/Premium): ")

fine = 10 if days <= 10 else 20 if days <= 20 else 50
total = fine * days

if member == "Premium":
    total *= 0.5

if days > 30:
    print("Membership Cancelled")
else:
    print("Total Fine:", total)
```

- **Output:**

```
Total Fine: 50
```

➢ Q13. Sales Analysis Dashboard:

- **Input:**

```
sales = [120, 130, 125, 140, 150, 145, 160, 170, 165, 180, 175, 190]

avg = sum(sales) / 12
print("Max:", max(sales))
print("Min:", min(sales))
print("Average:", avg)

for i in range(1, 12):
    if sales[i] > sales[i-1]:
        print("Month", i+1, ": Growth")
    else:
        print("Month", i+1, ": Decline")
```

- **Output:**

```
Max: 190
Min: 120
Average: 154.16666666666666
Month 2 : Growth
Month 3 : Decline
Month 4 : Growth
Month 5 : Growth
Month 6 : Decline
Month 7 : Growth
Month 8 : Growth
Month 9 : Decline
Month 10 : Growth
Month 11 : Decline
Month 12 : Growth
```

- **Q14. Multiplication Puzzle with Random Blanks:**
  - **Input:**

```
import random

score = 0
for i in range(1, 6):
    a = random.randint(1,10)
    b = random.randint(1,10)
    ans = a * b
    print(a, "x", b, "= ??")
    user = int(input("Guess: "))
    if user == ans:
        score += 1

print("Score:", score)
if score >= 4:
    print("Excellent")
elif score >= 2:
    print("Good")
else:
    print("Try Again")
```

  - **Output:**

```
8 x 3 = ??
Guess:  24
1 x 9 = ??
Guess:  9
6 x 9 = ??
Guess:  54
9 x 9 = ??
Guess:  82
8 x 5 = ??
Guess:  40
Score: 4
Excellent
```

- **Q15. Password Strength + Re-entry Attempts:**
  - **Input:**

```
attempts = 0

while attempts < 3:
    pwd = input("Enter Password: ")
    if (len(pwd) >= 8 and any(c.isupper() for c in pwd) and
        any(c.islower() for c in pwd) and any(c.isdigit() for c in pwd)):
        print("Strong Password")
        break
    else:
        print("Weak Password")
    attempts += 1

if attempts == 3:
    print("Account Locked")
```

  - **Output:**

```
Weak Password
Weak Password
Weak Password
Account Locked
```

- **Q16. Electricity Bill with Slabs & Penalty:**
  - **Input:**

```
def calculate_bill(units, late_payment=False):
    if units <= 100:
        bill = units * 5
    elif units <= 200:
        bill = units * 7
    else:
        bill = units * 10

    if late_payment:
        bill *= 2.78
    return bill

print("Bill:", calculate_bill(250, True))
```

- **Output:**

```
Bill: 6949.999999999999
```

➢ **Q17. Cricket Match Predictor with Multiple Conditions:**

- **Input:**

```
def predict_score(runs, balls, wickets, overs_left):
    run_rate = runs / (balls/6)

    if run_rate < 5 and wickets > 6:
        print("Losing")
    elif run_rate > 7 and wickets < 5:
        print("Winning")
    else:
        print("Balanced")

predict_score(150, 120, 4, 10)
```

- **Output:**

```
Winning
```

➢ **Q18. Hotel Booking with Tax & Discount:**

- **Input:**

```
def book_room(room, days, member):
    rates = {"Standard":2000, "Deluxe":4000, "Suite":6000}
    bill = rates[room] * days

    if days > 7:
        bill *= 0.9
    if member:
        bill *= 0.7

    bill *= 1.12
    return bill

print("Total Bill:", book_room("Deluxe", 8, True))
```

- **Output:**

```
Total Bill: 22579.2
```

➢ **Q19. Flight Fare Calculator with International Tax:**

- **Input:**

```
def calculate_fare(distance, class_type, international=False):
    rates = {"Economy":10, "Business":25, "First":40}
    fare = distance * rates[class_type] + 500

    if international:
        fare *= 2.22
    return fare

print("Fare:", calculate_fare(300, "Business", True))
```

- **Output:**

```
Fare: 17760.0
```

➢ **Q20. Smart Vending Machine with Wallet System:**

- **Input:**

```
def vending_machine(item, paid, wallet):
    items = {"Chips":50, "Soda":40, "Juice":60}

    if item not in items:
        print("Item not found")
    elif paid < items[item]:
        print("Insufficient amount")
    else:
        wallet -= items[item]
        print("Dispensed:", item)
        print("Change:", paid - items[item])
        print("Wallet Balance:", wallet)

vending_machine("Soda", 100, 500)
```

- **Output:**

```
Dispensed: Soda
Change: 60
Wallet Balance: 460
```

Lab 2

➢ OOP in Python:

- OOP stands for **Object-Oriented Programming**.
- Python supports OOP to organize code using **classes and objects**.
- A **class** is a blueprint, while an **object** is an instance of a class.
- OOP helps in **code reusability, readability, and maintenance**.
- Python supports key OOP concepts like **encapsulation, inheritance, polymorphism, and abstraction**.
- Using OOP makes large programs easier to manage and extend.

Question # 1: Create a `BankAccount` class that represents a bank account. It should have attributes for account number, account holder name, and balance. Include methods to deposit, withdraw, and display balance. Ensure withdrawals cannot exceed the available balance.

➢ Code:

```
class BankAccount:

    def __init__(self, account_number, holder_name, balance=0):

        self.account_number = account_number

        self.holder_name = holder_name

        self.balance = balance



    def deposit(self, amount):

        if amount > 0:

            self.balance += amount

            print(f"Deposited {amount}. New balance: {self.balance}")
```

```
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance! Withdrawal denied.")
        elif amount <= 0:
            print("Withdrawal amount must be positive.")
        else:
            self.balance -= amount
            print(f"Withdrew {amount}. New balance: {self.balance}")

    def display_balance(self):
        print(f"Account Holder: {self.holder_name}")
        print(f"Account Number: {self.account_number}")
        print(f"Current Balance: {self.balance}")


# Example usage
account1 = BankAccount("12345", "Abdul Haseeb", 1000)
account1.display_balance()
```

```
account1.deposit(500)

account1.withdraw(300)

account1.withdraw(1500)

account1.display_balance()
```

- **Output:**

```
Account Holder: Abdul Haseeb
Account Number: 12345
Current Balance: 1000
Deposited 500. New balance: 1500
Withdrew 300. New balance: 1200
Insufficient balance! Withdrawal denied.
Account Holder: Abdul Haseeb
Account Number: 12345
Current Balance: 1200
```

Question # 2: Design a `Book` class with attributes like title, author, ISBN, and availability status. Create a `Library` class that can add books, search for books by title/author, check out books, and return books. Track which books are currently available.

- **Code:**

```
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.available = True  # book is available by default

    def __str__(self):
        status = "Available" if self.available else "Checked Out"
        return f"'{self.title}' by {self.author} (ISBN: {self.isbn}) - {status}"


class Library:
```

```
def __init__(self):
    self.books = []

def add_book(self, book):
    self.books.append(book)
    print(f"Book added: {book.title} by {book.author}")

def search_by_title(self, title):
    print(f"\nSearching for books with title '{title}'...")
    found = [book for book in self.books if title.lower() in book.title.lower()]
    if found:
        for b in found:
            print(b)
    else:
        print("No book found with that title.")

def search_by_author(self, author):
    print(f"\nSearching for books by author '{author}'...")
    found = [book for book in self.books if author.lower() in book.author.lower()]
    if found:
        for b in found:
            print(b)
    else:
        print("No book found by that author.")

def check_out_book(self, isbn):
    for book in self.books:
        if book.isbn == isbn:
            if book.available:
                book.available = False
                print(f"You have checked out: {book.title}")
```

```
                return
            else:
                print("Sorry, this book is already checked out.")
                return
        print("Book not found.")

    def return_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                if not book.available:
                    book.available = True
                    print(f"Book returned: {book.title}")
                    return
                else:
                    print("This book was not checked out.")
                    return
        print("Book not found.")


# Example usage
library = Library()

# Adding books
book1 = Book("Python Basics", "John Smith", "1111")
book2 = Book("AI Fundamentals", "Abdul Haseeb", "2222")
book3 = Book("Learning C++", "Jane Doe", "3333")

library.add_book(book1)
library.add_book(book2)
library.add_book(book3)
```

```
# Searching books
library.search_by_title("Python")
library.search_by_author("Jane")

# Checking out and returning books
library.check_out_book("1111")
library.check_out_book("1111")  # Trying again (should show unavailable)
library.return_book("1111")
library.return_book("1111")  # Trying again (should show already returned)
```

- **Output:**

```
Book added: Python Basics by John Smith
Book added: AI Fundamentals by Abdul Haseeb
Book added: Learning C++ by Jane Doe

Searching for books with title 'Python'...
'Python Basics' by John Smith (ISBN: 1111) - Available

Searching for books by author 'Jane'...
'Learning C++' by Jane Doe (ISBN: 3333) - Available
You have checked out: Python Basics
Sorry, this book is already checked out.
Book returned: Python Basics
This book was not checked out.
```

Question # 3: Create a `Student` class with attributes for student ID, name, and a list of grades. Implement methods to add grades, calculate average grade, and determine the letter grade (A, B, C, etc.). Add a method to display student information with their performance summary.

- **Code:**

```
class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.grades = []

    def add_grade(self, grade):
        if 0 <= grade <= 100:
            self.grades.append(grade)
            print(f"Added grade {grade} for {self.name}")
        else:
            print("Grade must be between 0 and 100")

    def calculate_average(self):
        if len(self.grades) == 0:
            return 0
        return sum(self.grades) / len(self.grades)
```

```
    def get_letter_grade(self):
        avg = self.calculate_average()
        if avg >= 90:
            return "A"
        elif avg >= 80:
            return "B"
        elif avg >= 70:
            return "C"
        elif avg >= 60:
            return "D"
        else:
            return "F"

    def display_info(self):
        print("\n--- Student Information ---")
        print(f"Student ID: {self.student_id}")
        print(f"Name: {self.name}")
        print(f"Grades: {self.grades}")
        print(f"Average Grade: {self.calculate_average():.2f}")
        print(f"Letter Grade: {self.get_letter_grade()}")
```

```
        print("---------------------------")
```

```
# Example usage

student1 = Student("S001", "Abdul Haseeb")

student1.add_grade(85)

student1.add_grade(90)

student1.add_grade(78)

student1.display_info()
```

- **Output:**

```
Added grade 85 for Abdul Haseeb
Added grade 90 for Abdul Haseeb
Added grade 78 for Abdul Haseeb

--- Student Information ---
Student ID: S001
Name: Abdul Haseeb
Grades: [85, 90, 78]
Average Grade: 84.33
Letter Grade: B
---------------------------
```

Question # 4: Design a `MenuItem` class for food items (name, price, category) and an `Order` class that can add multiple items, calculate total cost, apply discounts, and generate receipts. Include tax calculation functionality.

- **Code:**

```
class MenuItem:
    def __init__(self, name, price, category):
        self.name = name
```

```
        self.price = price
        self.category = category

    def __str__(self):
        return f"{self.name} ({self.category}) - Rs.{self.price}"


class Order:
    def __init__(self):
        self.items = []
        self.tax_rate = 0.1  # 10% tax

    def add_item(self, item):
        self.items.append(item)
        print(f"Added: {item.name} - Rs.{item.price}")

    def calculate_total(self):
        subtotal = sum(item.price for item in self.items)
        tax = subtotal * self.tax_rate
        total = subtotal + tax
        return subtotal, tax, total

    def apply_discount(self, discount_percent):
        subtotal, tax, total = self.calculate_total()
        discount = total * (discount_percent / 100)
        total_after_discount = total - discount
        return total_after_discount

    def generate_receipt(self, discount_percent=0):
        print("\n------ RECEIPT ------")
        for item in self.items:
```

```
            print(f"{item.name} - Rs.{item.price}")

        subtotal, tax, total = self.calculate_total()
        print(f"\nSubtotal: Rs.{subtotal:.2f}")
        print(f"Tax (10%): Rs.{tax:.2f}")
        print(f"Total (with tax): Rs.{total:.2f}")

        if discount_percent > 0:
            total_after_discount = self.apply_discount(discount_percent)
            print(f"Discount: {discount_percent}%")
            print(f"Total after discount: Rs.{total_after_discount:.2f}")
        print("----------------------")


# Example usage
item1 = MenuItem("Burger", 250, "Fast Food")
item2 = MenuItem("Pizza", 800, "Fast Food")
item3 = MenuItem("Cola", 100, "Beverage")

order = Order()
order.add_item(item1)
order.add_item(item2)
order.add_item(item3)

order.generate_receipt(discount_percent=10)
```

- **Output:**

```
Added: Burger - Rs.250
Added: Pizza - Rs.800
Added: Cola - Rs.100

------ RECEIPT ------
Burger - Rs.250
Pizza - Rs.800
Cola - Rs.100

Subtotal: Rs.1150.00
Tax (10%): Rs.115.00
Total (with tax): Rs.1265.00
Discount: 10%
Total after discount: Rs.1138.50
--------------------
```

Question # 5: Create classes for `Vehicle` (base class) and derived classes `Car`, `Motorcycle`, and `Truck`. Each vehicle should have attributes like license plate, model, rental price per day, and availability. Implement a system to rent vehicles and calculate rental costs.

- **Code:**

```
# Base class
class Vehicle:
    def __init__(self, license_plate, model, price_per_day):
        self.license_plate = license_plate
        self.model = model
        self.price_per_day = price_per_day
        self.available = True


    def rent(self, days):
        if self.available:
            cost = self.price_per_day * days
            self.available = False
            print(f"{self.model} rented for {days} days. Total cost: Rs.{cost}")
        else:
            print(f"Sorry, {self.model} is not available right now.")
```

```
    def return_vehicle(self):
        if not self.available:
            self.available = True
            print(f"{self.model} has been returned and is now available.")
        else:
            print(f"{self.model} was not rented.")

    def display_info(self):
        status = "Available" if self.available else "Rented"
        print(f"{self.model} ({self.license_plate}) - Rs.{self.price_per_day}/day -
{status}")


# Derived classes
class Car(Vehicle):
    def __init__(self, license_plate, model, price_per_day, seats):
        super().__init__(license_plate, model, price_per_day)
        self.seats = seats

    def display_info(self):
        super().display_info()
        print(f"Type: Car | Seats: {self.seats}")


class Motorcycle(Vehicle):
    def __init__(self, license_plate, model, price_per_day, engine_cc):
        super().__init__(license_plate, model, price_per_day)
        self.engine_cc = engine_cc

    def display_info(self):
```

```
        super().display_info()
        print(f"Type: Motorcycle | Engine: {self.engine_cc}cc")


class Truck(Vehicle):
    def __init__(self, license_plate, model, price_per_day, capacity_tons):
        super().__init__(license_plate, model, price_per_day)
        self.capacity_tons = capacity_tons

    def display_info(self):
        super().display_info()
        print(f"Type: Truck | Capacity: {self.capacity_tons} tons")


# Example usage
car1 = Car("ABC-123", "Toyota Corolla", 5000, 5)
bike1 = Motorcycle("XYZ-456", "Honda 125", 1500, 125)
truck1 = Truck("LMN-789", "Hino Truck", 8000, 10)

# Display info
car1.display_info()
bike1.display_info()
truck1.display_info()

# Rent and return vehicles
car1.rent(3)
car1.return_vehicle()
bike1.rent(2)
truck1.rent(1)
```

- **Output:**

```
Toyota Corolla (ABC-123) - Rs.5000/day - Available
Type: Car | Seats: 5
Honda 125 (XYZ-456) - Rs.1500/day - Available
Type: Motorcycle | Engine: 125cc
Hino Truck (LMN-789) - Rs.8000/day - Available
Type: Truck | Capacity: 10 tons
Toyota Corolla rented for 3 days. Total cost: Rs.15000
Toyota Corolla has been returned and is now available.
Honda 125 rented for 2 days. Total cost: Rs.3000
Hino Truck rented for 1 days. Total cost: Rs.8000
```

Question # 6: Design an `Employee` class with attributes for employee ID, name, position, and salary. Create methods to calculate monthly salary, annual salary, and apply raises. Include functionality to track employee details and employment duration.

- **Code:**

```
from datetime import date


class Employee:
    def __init__(self, emp_id, name, position, salary, hire_year):
        self.emp_id = emp_id
        self.name = name
        self.position = position
        self.salary = salary      # Annual salary
        self.hire_year = hire_year

    def monthly_salary(self):
        return self.salary / 12

    def annual_salary(self):
        return self.salary

    def apply_raise(self, percent):
```

```
        if percent > 0:
            increase = self.salary * (percent / 100)
            self.salary += increase
            print(f"Salary increased by {percent}%. New annual salary: Rs.{self.salary:.2f}")
        else:
            print("Raise percentage must be positive.")

    def employment_duration(self):
        current_year = date.today().year
        years = current_year - self.hire_year
        return years

    def display_info(self):
        print("\n--- Employee Information ---")
        print(f"Employee ID: {self.emp_id}")
        print(f"Name: {self.name}")
        print(f"Position: {self.position}")
        print(f"Annual Salary: Rs.{self.salary:.2f}")
        print(f"Monthly Salary: Rs.{self.monthly_salary():.2f}")
        print(f"Employment Duration: {self.employment_duration()} years")
        print("-----------------------------")


# Example usage
emp1 = Employee("E001", "Abdul Haseeb", "AI Engineer", 1200000, 2020)
emp1.display_info()

emp1.apply_raise(10)
emp1.display_info()
```

- **Output:**

```
--- Employee Information ---
Employee ID: E001
Name: Abdul Haseeb
Position: AI Engineer
Annual Salary: Rs.1200000.00
Monthly Salary: Rs.100000.00
Employment Duration: 5 years
------------------------------
Salary increased by 10%. New annual salary: Rs.1320000.00

--- Employee Information ---
Employee ID: E001
Name: Abdul Haseeb
Position: AI Engineer
Annual Salary: Rs.1320000.00
Monthly Salary: Rs.110000.00
Employment Duration: 5 years
------------------------------
```

Question # 7: Create a `Product` class with attributes for product ID, name, price, and stock quantity. Design a `ShoppingCart` class that can add products, remove products, calculate total cost, and handle checkout process while updating inventory.

- **Code:**

```
class Product:

    def __init__(self, product_id, name, price, stock):

        self.product_id = product_id

        self.name = name

        self.price = price

        self.stock = stock
```

```
    def update_stock(self, quantity):
        if self.stock >= quantity:
            self.stock -= quantity
        else:
            print(f"Not enough stock for {self.name}!")

    def __str__(self):
        return f"{self.name} (Rs.{self.price}) - Stock: {self.stock}"


class ShoppingCart:
    def __init__(self):
        self.items = {}

    def add_product(self, product, quantity):
        if product.stock >= quantity:
            if product.product_id in self.items:
                self.items[product.product_id]['quantity'] += quantity
            else:
                self.items[product.product_id] = {'product': product, 'quantity': quantity}
            product.update_stock(quantity)
```

```
            print(f"Added {quantity} x {product.name} to cart.")
        else:
            print(f"Sorry, only {product.stock} items left in stock.")

    def remove_product(self, product_id):
        if product_id in self.items:
            product = self.items[product_id]['product']
            quantity = self.items[product_id]['quantity']
            product.stock += quantity  # return stock
            del self.items[product_id]
            print(f"Removed {product.name} from cart.")
        else:
            print("Product not found in cart.")

    def calculate_total(self):
        total = sum(item['product'].price * item['quantity'] for item in self.items.values())
        return total

    def checkout(self):
        if not self.items:
            print("Your cart is empty.")
```

```
            return
        total = self.calculate_total()
        print("\n----- CHECKOUT RECEIPT -----")
        for item in self.items.values():
            product = item['product']
            quantity = item['quantity']
            print(f"{product.name} x{quantity} = Rs.{product.price * quantity}")
        print(f"Total Amount: Rs.{total}")
        print("-----------------------------")
        self.items.clear()
        print("Thank you for your purchase!")


# Example usage
p1 = Product("P001", "Laptop", 80000, 5)
p2 = Product("P002", "Headphones", 2500, 10)
p3 = Product("P003", "Mouse", 1200, 8)

cart = ShoppingCart()

print("\n--- Available Products ---")
```

```
print(p1)

print(p2)

print(p3)


cart.add_product(p1, 1)

cart.add_product(p2, 2)

cart.add_product(p3, 1)


cart.remove_product("P002")  # Optional: remove one product


cart.checkout()


print("\n--- Updated Stock After Checkout ---")

print(p1)

print(p2)

print(p3)
```

- **Output:**

```
--- Available Products ---
Laptop (Rs.80000) - Stock: 5
Headphones (Rs.2500) - Stock: 10
Mouse (Rs.1200) - Stock: 8
Added 1 x Laptop to cart.
Added 2 x Headphones to cart.
Added 1 x Mouse to cart.
Removed Headphones from cart.

----- CHECKOUT RECEIPT -----
Laptop x1 = Rs.80000
Mouse x1 = Rs.1200
Total Amount: Rs.81200
----------------------------
Thank you for your purchase!

--- Updated Stock After Checkout ---
Laptop (Rs.80000) - Stock: 4
Headphones (Rs.2500) - Stock: 10
Mouse (Rs.1200) - Stock: 7
```

Question # 8: Design a `Patient` class to store patient information (ID, name, age, medical history) and an `Appointment` class to manage doctor appointments. Include functionality to schedule, cancel, and view appointments.

- **Code:**

```
class Patient:
    def __init__(self, patient_id, name, age, medical_history=None):
        self.patient_id = patient_id
        self.name = name
        self.age = age
        self.medical_history = medical_history if medical_history else []
```

```
    def add_medical_record(self, record):
        self.medical_history.append(record)
        print(f"Medical record added for {self.name}.")

    def display_info(self):
        print("\n--- Patient Information ---")
        print(f"Patient ID: {self.patient_id}")
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")
        print("Medical History:")
        for record in self.medical_history:
            print(f" - {record}")
        print("----------------------------")


class Appointment:
    def __init__(self):
        self.appointments = []

    def schedule_appointment(self, patient, doctor_name, date, time):
        appointment = {
            "patient": patient.name,
            "doctor": doctor_name,
            "date": date,
            "time": time
        }
        self.appointments.append(appointment)
        print(f"Appointment scheduled for {patient.name} with Dr. {doctor_name}
on {date} at {time}.")

    def cancel_appointment(self, patient_name, doctor_name):
```

```
        for a in self.appointments:
            if a["patient"] == patient_name and a["doctor"] == doctor_name:
                self.appointments.remove(a)
                print(f"Appointment for {patient_name} with Dr. {doctor_name} has been canceled.")
                return
        print("No matching appointment found to cancel.")

    def view_appointments(self):
        if not self.appointments:
            print("No appointments scheduled.")
            return
        print("\n--- Appointments List ---")
        for a in self.appointments:
            print(f"Patient: {a['patient']} | Doctor: {a['doctor']} | Date: {a['date']} | Time: {a['time']}")
        print("--------------------------")


# Example usage
patient1 = Patient("P001", "Abdul Haseeb", 25, ["Flu", "Headache"])
patient1.display_info()

appointment_system = Appointment()

# Schedule appointments
appointment_system.schedule_appointment(patient1, "Dr. Ali", "2025-10-20", "10:00 AM")
appointment_system.schedule_appointment(patient1, "Dr. Sara", "2025-10-25", "02:00 PM")
```

```
# View all appointments
appointment_system.view_appointments()

# Cancel one appointment
appointment_system.cancel_appointment("Abdul Haseeb", "Dr. Sara")

# View updated appointments
appointment_system.view_appointments()
```

- **Output:**

```
--- Patient Information ---
Patient ID: P001
Name: Abdul Haseeb
Age: 25
Medical History:
 - Flu
 - Headache
----------------------------
Appointment scheduled for Abdul Haseeb with Dr. Dr. Ali on 2025-10-20 at 10:00 AM.
Appointment scheduled for Abdul Haseeb with Dr. Dr. Sara on 2025-10-25 at 02:00 PM.

--- Appointments List ---
Patient: Abdul Haseeb | Doctor: Dr. Ali | Date: 2025-10-20 | Time: 10:00 AM
Patient: Abdul Haseeb | Doctor: Dr. Sara | Date: 2025-10-25 | Time: 02:00 PM
--------------------------
Appointment for Abdul Haseeb with Dr. Dr. Sara has been canceled.

--- Appointments List ---
Patient: Abdul Haseeb | Doctor: Dr. Ali | Date: 2025-10-20 | Time: 10:00 AM
--------------------------
```

Question # 9: Create a `User` class for a social media platform with attributes for username, email, friends list, and posts. Implement methods to add friends, create posts, and display user timeline. Include privacy settings for posts.

- **Code:**

```
class User:
    def __init__(self, username, email):
        self.username = username
        self.email = email
        self.friends = []
        self.posts = []  # Each post will have content and privacy

    def add_friend(self, friend_name):
```

```
        if friend_name not in self.friends:
            self.friends.append(friend_name)
            print(f"{friend_name} added as a friend.")
        else:
            print(f"{friend_name} is already your friend.")

    def create_post(self, content, privacy="Public"):
        post = {"content": content, "privacy": privacy}
        self.posts.append(post)
        print(f"Post created with {privacy} privacy.")

    def display_timeline(self, viewer_name=None):
        print(f"\n--- {self.username}'s Timeline ---")
        if not self.posts:
            print("No posts yet.")
        for post in self.posts:
            if post["privacy"] == "Public":
                print(f"[Public] {post['content']}")
            elif post["privacy"] == "Friends Only" and viewer_name in self.friends:
                print(f"[Friends Only] {post['content']}")
            elif post["privacy"] == "Private" and viewer_name == self.username:
                print(f"[Private] {post['content']}")
        print("------------------------------")

    def display_info(self):
        print(f"\nUser: {self.username}")
        print(f"Email: {self.email}")
        print(f"Friends: {', '.join(self.friends) if self.friends else 'No friends yet.'}")


# Example usage
```

```
user1 = User("AbdulHaseeb", "haseeb@example.com")
user2 = User("AliRaza", "ali@example.com")

user1.add_friend("AliRaza")

user1.create_post("Hello everyone! This is my first post.", "Public")
user1.create_post("Having lunch with friends.", "Friends Only")
user1.create_post("My private thoughts.", "Private")

# View timelines
user1.display_info()
user1.display_timeline(viewer_name="AliRaza")  # Ali can see public and friends-only posts
user1.display_timeline(viewer_name="AbdulHaseeb")  # Owner sees all posts
```

- **Output:**

```
AliRaza added as a friend.
Post created with Public privacy.
Post created with Friends Only privacy.
Post created with Private privacy.

User: AbdulHaseeb
Email: haseeb@example.com
Friends: AliRaza

--- AbdulHaseeb's Timeline ---
[Public] Hello everyone! This is my first post.
[Friends Only] Having lunch with friends.
------------------------------

--- AbdulHaseeb's Timeline ---
[Public] Hello everyone! This is my first post.
[Private] My private thoughts.
------------------------------
```

Question # 10: Design a `Room` class with room number, type (single/double/suite), price, and availability. Create a `Booking` class to handle room reservations, check-in/check-out dates, and calculate total stay cost.

- **Code:**

```
class Room:
    def __init__(self, room_number, room_type, price):
        self.room_number = room_number
        self.room_type = room_type
        self.price = price
        self.available = True

    def display_info(self):
        status = "Available" if self.available else "Booked"
        print(f'Room {self.room_number} | Type: {self.room_type} | Price: Rs.{self.price}/night | {status}")


class Booking:
    def __init__(self):
        self.bookings = []

    def book_room(self, room, guest_name, check_in_date, check_out_date, nights):
        if room.available:
            total_cost = room.price * nights
            booking_details = {
                "guest": guest_name,
                "room_number": room.room_number,
                "room_type": room.room_type,
                "check_in": check_in_date,
                "check_out": check_out_date,
```

```
            "nights": nights,
            "total_cost": total_cost
        }
        self.bookings.append(booking_details)
        room.available = False
        print(f"Room {room.room_number} booked for {guest_name} from {check_in_date} to {check_out_date}.")
        print(f"Total cost: Rs.{total_cost}")
    else:
        print(f"Room {room.room_number} is not available right now.")

def checkout_room(self, room_number):
    for booking in self.bookings:
        if booking["room_number"] == room_number:
            self.bookings.remove(booking)
            print(f"Guest {booking['guest']} has checked out of Room {room_number}.")
            return booking
    print("No booking found for that room number.")

def display_bookings(self):
    if not self.bookings:
        print("No current bookings.")
        return
    print("\n--- Current Bookings ---")
    for b in self.bookings:
        print(f"Guest: {b['guest']} | Room: {b['room_number']} ({b['room_type']}) | "
              f"Check-in: {b['check_in']} | Check-out: {b['check_out']} | Total: Rs.{b['total_cost']}")
    print("-------------------------")
```

```
# Example usage
room1 = Room(101, "Single", 4000)
room2 = Room(102, "Double", 6000)
room3 = Room(201, "Suite", 10000)

# Display available rooms
print("\n--- Available Rooms ---")
room1.display_info()
room2.display_info()
room3.display_info()

# Create booking system
hotel_booking = Booking()

# Book a room
hotel_booking.book_room(room1, "Abdul Haseeb", "2025-10-15", "2025-10-18",
3)
hotel_booking.book_room(room2, "Ali Raza", "2025-10-16", "2025-10-19", 3)

# View all bookings
hotel_booking.display_bookings()

# Checkout a room
hotel_booking.checkout_room(101)
room1.available = True  # make the room available again

# View updated bookings and room status
hotel_booking.display_bookings()
print("\n--- Updated Room Info ---")
```

```
room1.display_info()
```

- **Output:**

```
--- Available Rooms ---
Room 101 | Type: Single | Price: Rs.4000/night | Available
Room 102 | Type: Double | Price: Rs.6000/night | Available
Room 201 | Type: Suite | Price: Rs.10000/night | Available
Room 101 booked for Abdul Haseeb from 2025-10-15 to 2025-10-18.
Total cost: Rs.12000
Room 102 booked for Ali Raza from 2025-10-16 to 2025-10-19.
Total cost: Rs.18000

--- Current Bookings ---
Guest: Abdul Haseeb | Room: 101 (Single) | Check-in: 2025-10-15 | Check-out: 2025-10-18 | Total: Rs.12000
Guest: Ali Raza | Room: 102 (Double) | Check-in: 2025-10-16 | Check-out: 2025-10-19 | Total: Rs.18000
-------------------------
Guest Abdul Haseeb has checked out of Room 101.

--- Current Bookings ---
Guest: Ali Raza | Room: 102 (Double) | Check-in: 2025-10-16 | Check-out: 2025-10-19 | Total: Rs.18000
-------------------------

--- Updated Room Info ---
Room 101 | Type: Single | Price: Rs.4000/night | Available
```

Question # 11: Create `Course` classes (with code, name, credits, capacity) and a `Student` class that can register for courses. Implement functionality to check for schedule conflicts and ensure students don't exceed credit limits.

- **Code:**

```
class Course:

    def __init__(self, code, name, credits, capacity, schedule):

        self.code = code

        self.name = name

        self.credits = credits

        self.capacity = capacity

        self.schedule = schedule  # e.g. "Mon 10-12"

        self.enrolled_students = []
```

```
    def add_student(self, student_name):
        if len(self.enrolled_students) < self.capacity:
            self.enrolled_students.append(student_name)
            print(f"{student_name} enrolled in {self.name}.")
        else:
            print(f"Course {self.name} is full!")

    def display_info(self):
        print(f"{self.code}: {self.name} | Credits: {self.credits} | "
              f"Schedule: {self.schedule} | Capacity: {len(self.enrolled_students)}/{self.capacity}")


class Student:
    def __init__(self, student_id, name, max_credits=9):
        self.student_id = student_id
        self.name = name
        self.max_credits = max_credits
        self.registered_courses = []

    def total_credits(self):
```

```
        return sum(course.credits for course in self.registered_courses)


    def has_schedule_conflict(self, new_course):
        for course in self.registered_courses:
            if course.schedule == new_course.schedule:
                return True
        return False


    def register_course(self, course):
        if self.has_schedule_conflict(course):
            print(f"Cannot register {course.name}: Schedule conflict with another course.")
            return


        if self.total_credits() + course.credits > self.max_credits:
            print(f"Cannot register {course.name}: Credit limit exceeded.")
            return


        if len(course.enrolled_students) >= course.capacity:
            print(f"Cannot register {course.name}: Course is full.")
            return
```

```
        self.registered_courses.append(course)
        course.add_student(self.name)
        print(f"{self.name} successfully registered for {course.name}.")

    def display_registered_courses(self):
        print(f"\n--- {self.name}'s Registered Courses ---")
        if not self.registered_courses:
            print("No courses registered.")
        for course in self.registered_courses:
            print(f"{course.code} - {course.name} ({course.credits} credits) | {course.schedule}")
        print("----------------------------------------")


# Example usage
c1 = Course("CS101", "Intro to Programming", 3, 2, "Mon 10-12")
c2 = Course("AI201", "Artificial Intelligence", 3, 2, "Mon 10-12")
c3 = Course("DS102", "Data Science", 4, 2, "Wed 2-4")

s1 = Student("S001", "Abdul Haseeb")
```

```
s2 = Student("S002", "Ali Raza")


# Register students for courses
s1.register_course(c1)
s1.register_course(c2)  # Schedule conflict
s1.register_course(c3)  # Within credit limit


s2.register_course(c1)
s2.register_course(c3)


# Display registered courses
s1.display_registered_courses()
s2.display_registered_courses()


# Display course info
print("\n--- Course Information ---")
c1.display_info()
c2.display_info()
c3.display_info()
```

- **Output:**

```
Abdul Haseeb enrolled in Intro to Programming.
Abdul Haseeb successfully registered for Intro to Programming.
Cannot register Artificial Intelligence: Schedule conflict with another course.
Abdul Haseeb enrolled in Data Science.
Abdul Haseeb successfully registered for Data Science.
Ali Raza enrolled in Intro to Programming.
Ali Raza successfully registered for Intro to Programming.
Ali Raza enrolled in Data Science.
Ali Raza successfully registered for Data Science.

--- Abdul Haseeb's Registered Courses ---
CS101 - Intro to Programming (3 credits) | Mon 10-12
DS102 - Data Science (4 credits) | Wed 2-4
----------------------------------------

--- Ali Raza's Registered Courses ---
CS101 - Intro to Programming (3 credits) | Mon 10-12
DS102 - Data Science (4 credits) | Wed 2-4
----------------------------------------

--- Course Information ---
CS101: Intro to Programming | Credits: 3 | Schedule: Mon 10-12 | Capacity: 2/2
AI201: Artificial Intelligence | Credits: 3 | Schedule: Mon 10-12 | Capacity: 0/2
DS102: Data Science | Credits: 4 | Schedule: Wed 2-4 | Capacity: 2/2
```

Question # 12: Design a `Product` class for items in inventory (ID, name, quantity, price, supplier) and an `Inventory` class to track stock levels, add new products, update quantities, and generate low-stock alerts.

- **Code:**

```
class Product:
    def __init__(self, product_id, name, quantity, price, supplier):
        self.product_id = product_id
        self.name = name
        self.quantity = quantity
        self.price = price
        self.supplier = supplier
```

```
    def display_info(self):
        print(f"ID: {self.product_id}, Name: {self.name}, Quantity: {self.quantity}, "
              f"Price: ${self.price}, Supplier: {self.supplier}")


class Inventory:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)
        print(f"Product '{product.name}' added to inventory.")

    def update_quantity(self, product_id, new_quantity):
        for product in self.products:
            if product.product_id == product_id:
                product.quantity = new_quantity
                print(f"Updated quantity of '{product.name}' to {new_quantity}.")
                return
        print("Product not found.")

    def display_inventory(self):
        print("\n--- Inventory List ---")
        if not self.products:
            print("No products in inventory.")
        for product in self.products:
            product.display_info()

    def low_stock_alert(self, limit=5):
        print("\n--- Low Stock Alert ---")
        low_stock = [p for p in self.products if p.quantity <= limit]
```

```
        if not low_stock:
            print("All products are sufficiently stocked.")
        else:
            for product in low_stock:
                print(f"⚠ Low stock: {product.name} (Qty: {product.quantity})")


# Example usage
p1 = Product(1, "Laptop", 3, 1200, "TechWorld")
p2 = Product(2, "Mouse", 10, 25, "GadgetCo")
p3 = Product(3, "Keyboard", 2, 45, "KeyMasters")

inventory = Inventory()
inventory.add_product(p1)
inventory.add_product(p2)
inventory.add_product(p3)

inventory.display_inventory()
inventory.low_stock_alert(limit=5)

inventory.update_quantity(3, 8)  # Update keyboard stock
inventory.display_inventory()
inventory.low_stock_alert(limit=5)
```

- **Output:**

```
Product 'Laptop' added to inventory.
Product 'Mouse' added to inventory.
Product 'Keyboard' added to inventory.

--- Inventory List ---
ID: 1, Name: Laptop, Quantity: 3, Price: $1200, Supplier: TechWorld
ID: 2, Name: Mouse, Quantity: 10, Price: $25, Supplier: GadgetCo
ID: 3, Name: Keyboard, Quantity: 2, Price: $45, Supplier: KeyMasters

--- Low Stock Alert ---
⚠️ Low stock: Laptop (Qty: 3)
⚠️ Low stock: Keyboard (Qty: 2)
Updated quantity of 'Keyboard' to 8.

--- Inventory List ---
ID: 1, Name: Laptop, Quantity: 3, Price: $1200, Supplier: TechWorld
ID: 2, Name: Mouse, Quantity: 10, Price: $25, Supplier: GadgetCo
ID: 3, Name: Keyboard, Quantity: 8, Price: $45, Supplier: KeyMasters

--- Low Stock Alert ---
⚠️ Low stock: Laptop (Qty: 3)
```

Question # 13: Create `Movie` classes (title, genre, duration, showtimes) and a `Theater` class with seating arrangement. Implement a booking system that reserves seats and calculates ticket prices with different categories (standard, premium).

- **Code:**

```
class Movie:
    def __init__(self, title, genre, duration, showtimes):
        self.title = title
        self.genre = genre
        self.duration = duration  # in minutes
        self.showtimes = showtimes  # e.g. ["12:00 PM", "3:00 PM", "6:00 PM"]

    def display_info(self):
        print(f"🎬 {self.title} | Genre: {self.genre} | Duration: {self.duration} min")
```

```
        print(f"Showtimes: {', '.join(self.showtimes)}\n")


class Theater:
    def __init__(self, name, rows=5, seats_per_row=5):
        self.name = name
        self.seating = [["O" for _ in range(seats_per_row)] for _ in range(rows)]  # O
= Open seat
        self.ticket_prices = {"standard": 500, "premium": 800}

    def display_seating(self):
        print(f"\n Seating Layout - {self.name}")
        for row in self.seating:
            print(" ".join(row))
        print("O = Available | X = Booked")

    def book_seat(self, row, seat, category="standard"):
        if row < 0 or row >= len(self.seating) or seat < 0 or seat >= len(self.seating[0]):
            print("Invalid seat number!")
            return

        if self.seating[row][seat] == "X":
            print("Seat already booked!")
        else:
            self.seating[row][seat] = "X"
            price = self.ticket_prices.get(category.lower(), 500)
            print(f" Seat booked successfully! Ticket Price: Rs. {price}")

    def available_seats(self):
        count = sum(row.count("O") for row in self.seating)
        print(f"Available Seats: {count}")
```

```
# Example Usage
movie1 = Movie("Avengers: Endgame", "Action", 180, ["12:00 PM", "6:00 PM"])
movie2 = Movie("Inside Out 2", "Animation", 95, ["1:00 PM", "4:00 PM"])

theater = Theater("CineStar Theater", rows=4, seats_per_row=6)

# Display movie details
movie1.display_info()
movie2.display_info()

# Show seating and book tickets
theater.display_seating()
theater.book_seat(1, 2, "premium")  # Booking seat in row 1, seat 2
theater.book_seat(1, 2, "standard")  # Trying same seat again
theater.book_seat(2, 3, "standard")

theater.display_seating()
theater.available_seats()
```

➢ **Output:**

```
🎬 Avengers: Endgame | Genre: Action | Duration: 180 min
Showtimes: 12:00 PM, 6:00 PM

🎬 Inside Out 2 | Genre: Animation | Duration: 95 min
Showtimes: 1:00 PM, 4:00 PM


🎟️ Seating Layout - CineStar Theater
O O O O O O
O O O O O O
O O O O O O
O O O O O O
O = Available | X = Booked
✅ Seat booked successfully! Ticket Price: Rs. 800
Seat already booked!
✅ Seat booked successfully! Ticket Price: Rs. 500

🎟️ Seating Layout - CineStar Theater
O O O O O O
O O X O O O
O O O X O O
O O O O O O
O = Available | X = Booked
Available Seats: 22
```

Question # 14: Design a `Member` class with personal details, membership type (basic/premium), and attendance records. Create a `Trainer` class and a system to schedule training sessions between members and trainers.

➢ **Code:**

```
class Member:
    def __init__(self, member_id, name, membership_type):
        self.member_id = member_id
        self.name = name
        self.membership_type = membership_type  # "basic" or "premium"
        self.attendance = []  # list of attended session dates

    def mark_attendance(self, date):
        self.attendance.append(date)
        print(f"✅ Attendance marked for {self.name} on {date}")

    def display_info(self):
        print(f"ID: {self.member_id} | Name: {self.name} | Type: {self.membership_type}")
        print(f"Attendance: {len(self.attendance)} sessions\n")
```

```
class Trainer:
    def __init__(self, trainer_id, name, specialty):
        self.trainer_id = trainer_id
        self.name = name
        self.specialty = specialty
        self.schedule = []  # list of sessions

    def schedule_session(self, member, date, time):
        session = {
            "member": member.name,
            "date": date,
            "time": time
        }
        self.schedule.append(session)
        print(f"📅 Session scheduled: {member.name} with Trainer {self.name} on {date} at {time}")

    def display_schedule(self):
        print(f"\nTrainer: {self.name} | Specialty: {self.specialty}")
        if not self.schedule:
            print("No sessions scheduled.")
        else:
            for s in self.schedule:
                print(f"- {s['date']} at {s['time']} with {s['member']}")


# Example usage
m1 = Member(1, "Abdul Haseeb", "premium")
m2 = Member(2, "Ali Raza", "basic")
```

```
t1 = Trainer(101, "Sarah Khan", "Fitness")
t2 = Trainer(102, "Ahmed Malik", "Yoga")

# Schedule training sessions
t1.schedule_session(m1, "2025-10-15", "10:00 AM")
t2.schedule_session(m2, "2025-10-16", "5:00 PM")

# Mark attendance
m1.mark_attendance("2025-10-15")

# Display information
print("\n--- Members Info ---")
m1.display_info()
m2.display_info()

print("\n--- Trainers Schedule ---")
t1.display_schedule()
t2.display_schedule()
```

- **Output:**

```
Session scheduled: Abdul Haseeb with Trainer Sarah Khan on 2025-10-15 at 10:00 AM
Session scheduled: Ali Raza with Trainer Ahmed Malik on 2025-10-16 at 5:00 PM
✅ Attendance marked for Abdul Haseeb on 2025-10-15

--- Members Info ---
ID: 1 | Name: Abdul Haseeb | Type: premium
Attendance: 1 sessions

ID: 2 | Name: Ali Raza | Type: basic
Attendance: 0 sessions


--- Trainers Schedule ---

Trainer: Sarah Khan | Specialty: Fitness
- 2025-10-15 at 10:00 AM with Abdul Haseeb

Trainer: Ahmed Malik | Specialty: Yoga
- 2025-10-16 at 5:00 PM with Ali Raza
```

Question # 15: Extend the bank account system to include `SavingsAccount` and `CheckingAccount` classes with different interest rates and withdrawal rules. Implement fund transfer between accounts.

- **Code:**

```
# Base Class
class BankAccount:
    def __init__(self, account_number, holder_name, balance=0):
        self.account_number = account_number
        self.holder_name = holder_name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"✅ Deposited Rs.{amount}. New Balance: Rs.{self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("❌ Not enough balance!")
        else:
            self.balance -= amount
            print(f"💸 Withdrawn Rs.{amount}. Remaining Balance: Rs.{self.balance}")

    def transfer(self, other_account, amount):
        if amount > self.balance:
            print("❌ Transfer failed! Not enough balance.")
        else:
            self.balance -= amount
            other_account.balance += amount
            print(f"✅ Rs.{amount} transferred to {other_account.holder_name}")
```

```
    def show_balance(self):
        print(f"Account: {self.account_number} | Holder: {self.holder_name} | Balance: Rs.{self.balance}")


# Savings Account Class
class SavingsAccount(BankAccount):
    def __init__(self, account_number, holder_name, balance=0, interest_rate=0.04):
        super().__init__(account_number, holder_name, balance)
        self.interest_rate = interest_rate

    def add_interest(self):
        interest = self.balance * self.interest_rate
        self.balance += interest
        print(f"💰 Interest of Rs.{interest:.2f} added. New Balance: Rs.{self.balance:.2f}")


# Checking Account Class
class CheckingAccount(BankAccount):
    def __init__(self, account_number, holder_name, balance=0, overdraft_limit=5000):
        super().__init__(account_number, holder_name, balance)
        self.overdraft_limit = overdraft_limit

    def withdraw(self, amount):
        if amount > self.balance + self.overdraft_limit:
            print("❌ Withdrawal exceeds overdraft limit!")
        else:
```

```
        self.balance -= amount
        print(f"💸 Withdrawn Rs.{amount}. Remaining Balance: Rs.{self.balance}")


# Example usage
s1 = SavingsAccount("SA101", "Abdul Haseeb", 10000)
c1 = CheckingAccount("CA102", "Ali Raza", 5000)

print("\n--- Initial Account Balances ---")
s1.show_balance()
c1.show_balance()

# Deposit, Withdraw, Interest, and Transfer
s1.deposit(2000)
s1.add_interest()
c1.withdraw(6000)
s1.transfer(c1, 3000)

print("\n--- Final Account Balances ---")
s1.show_balance()
c1.show_balance()
```

- **Output:**

```
--- Initial Account Balances ---
Account: SA101 | Holder: Abdul Haseeb | Balance: Rs.10000
Account: CA102 | Holder: Ali Raza | Balance: Rs.5000
✅ Deposited Rs.2000. New Balance: Rs.12000
💰 Interest of Rs.480.00 added. New Balance: Rs.12480.00
💸 Withdrawn Rs.6000. Remaining Balance: Rs.-1000
✅ Rs.3000 transferred to Ali Raza

--- Final Account Balances ---
Account: SA101 | Holder: Abdul Haseeb | Balance: Rs.9480.0
Account: CA102 | Holder: Ali Raza | Balance: Rs.2000
```

Question # 16: Create a `Pizza` class with size, crust type, and toppings. Design an ordering system that calculates prices based on selections and allows custom pizza creation with various topping combinations.

- **Code:**

```
class Pizza:
    def __init__(self, size, crust, toppings):
        self.size = size          # small, medium, large
        self.crust = crust        # thin, thick, cheese burst
        self.toppings = toppings  # list of toppings

    def calculate_price(self):
        # Base price by size
        size_prices = {"small": 500, "medium": 800, "large": 1100}
        crust_prices = {"thin": 100, "thick": 150, "cheese burst": 200}
        topping_price = 50 * len(self.toppings)

        total = size_prices.get(self.size.lower(), 0) + \
                crust_prices.get(self.crust.lower(), 0) + topping_price
        return total

    def display_order(self):
        print(f"\n🍕 Pizza Order Summary:")
        print(f"Size: {self.size}")
        print(f"Crust: {self.crust}")
        print(f"Toppings: {', '.join(self.toppings) if self.toppings else 'None'}")
        print(f"Total Price: Rs.{self.calculate_price()}")


# Example Usage
print("Welcome to Python Pizza Shop! 🍕")
```

```
# Choose pizza size
size = input("Enter size (Small/Medium/Large): ")

# Choose crust type
crust = input("Enter crust type (Thin/Thick/Cheese Burst): ")

# Choose toppings
print("Available toppings: Olives, Mushrooms, Pepperoni, Onions, Extra Cheese")
toppings_input = input("Enter toppings separated by commas (or press Enter for none): ")
toppings = [t.strip().capitalize() for t in toppings_input.split(",") if t.strip()]

# Create Pizza object
pizza = Pizza(size, crust, toppings)

# Display order summary
pizza.display_order()
```

- **Output:**

```
Welcome to Python Pizza Shop! 🍕
Enter size (Small/Medium/Large):  medium
Enter crust type (Thin/Thick/Cheese Burst):  cheese burst
Available toppings: Olives, Mushrooms, Pepperoni, Onions, Extra Cheese
Enter toppings separated by commas (or press Enter for none):  mushrooms

🍕 Pizza Order Summary:
Size: medium
Crust: cheese burst
Toppings: Mushrooms
Total Price: Rs.1050
```

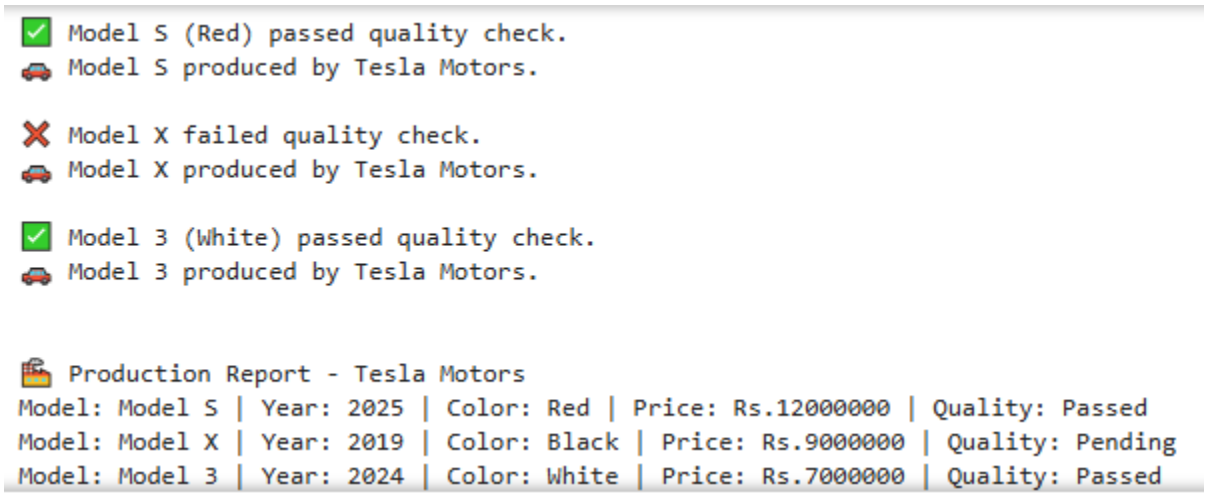
Question # 17: Design classes for `Car` (model, year, color, price) and `CarManufacturer` that can produce different car models with various features. Include quality control checks and production tracking.

- **Code:**

```
class Car:
    def __init__(self, model, year, color, price):
        self.model = model
        self.year = year
        self.color = color
        self.price = price
        self.passed_quality_check = False

    def quality_check(self):
        # Simple quality check logic
        if self.price > 0 and self.year >= 2020:
            self.passed_quality_check = True
            print(f"✅ {self.model} ({self.color}) passed quality check.")
        else:
            print(f"❌ {self.model} failed quality check.")

    def display_info(self):
        status = "Passed" if self.passed_quality_check else "Pending"
        print(f"Model: {self.model} | Year: {self.year} | Color: {self.color} | "
              f"Price: Rs.{self.price} | Quality: {status}")


class CarManufacturer:
    def __init__(self, name):
        self.name = name
        self.production_list = []

    def produce_car(self, model, year, color, price):
        car = Car(model, year, color, price)
        car.quality_check()
```

```
        self.production_list.append(car)
        print(f"🚗 {model} produced by {self.name}.\n")
        return car

    def show_production_report(self):
        print(f"\n🏭 Production Report - {self.name}")
        if not self.production_list:
            print("No cars produced yet.")
        else:
            for car in self.production_list:
                car.display_info()


# Example usage
manufacturer = CarManufacturer("Tesla Motors")

# Produce some cars
manufacturer.produce_car("Model S", 2025, "Red", 12000000)
manufacturer.produce_car("Model X", 2019, "Black", 9000000)  # Fails QC
manufacturer.produce_car("Model 3", 2024, "White", 7000000)

# Display production report
manufacturer.show_production_report()
```

- **Output:**



Question # 18: Create `Teacher` and `Student` classes, and a `Classroom` class that manages assignments, grades, and attendance. Implement functionality to track student performance and generate class reports.

- **Code:**

```
class Teacher:
    def __init__(self, name, subject):
        self.name = name
        self.subject = subject

    def display_info(self):
        print(f"👩‍🏫 Teacher: {self.name} | Subject: {self.subject}")



class Student:
    def __init__(self, student_id, name):
        self.student_id = student_id
        self.name = name
        self.grades = {}     # {assignment: grade}
        self.attendance = 0  # total days attended
```

```
    def mark_attendance(self):
        self.attendance += 1
        print(f"✅ Attendance marked for {self.name}")

    def add_grade(self, assignment, grade):
        self.grades[assignment] = grade
        print(f"📝 {self.name} got {grade}% on {assignment}")

    def average_grade(self):
        if self.grades:
            return sum(self.grades.values()) / len(self.grades)
        return 0

    def display_info(self):
        avg = self.average_grade()
        print(f"🎓 {self.name} | Attendance: {self.attendance} days | Average Grade: {avg:.1f}%")


class Classroom:
    def __init__(self, teacher):
        self.teacher = teacher
        self.students = []

    def add_student(self, student):
        self.students.append(student)
        print(f"👨‍🎓 {student.name} added to the class.")

    def mark_attendance(self):
        print("\n📅 Marking attendance for all students...")
```

```
        for student in self.students:
            student.mark_attendance()

    def add_assignment_grades(self, assignment_name, grades_dict):
        print(f"\n Recording grades for '{assignment_name}'")
        for student in self.students:
            if student.name in grades_dict:
                student.add_grade(assignment_name, grades_dict[student.name])

    def generate_report(self):
        print("\n --- CLASS REPORT ---")
        self.teacher.display_info()
        for student in self.students:
            student.display_info()


# Example usage
teacher = Teacher("Sarah Khan", "Mathematics")
classroom = Classroom(teacher)

# Add students
s1 = Student(1, "Abdul Haseeb")
s2 = Student(2, "Ali Raza")
classroom.add_student(s1)
classroom.add_student(s2)

# Mark attendance
classroom.mark_attendance()

# Add grades for an assignment
grades = {"Abdul Haseeb": 85, "Ali Raza": 90}
```

```
classroom.add_assignment_grades("Algebra Test", grades)


# Generate report
classroom.generate_report()
```

- **Output:**

```
👨‍🎓 Abdul Haseeb added to the class.
👨‍🎓 Ali Raza added to the class.

📅 Marking attendance for all students...
✅ Attendance marked for Abdul Haseeb
✅ Attendance marked for Ali Raza

📊 Recording grades for 'Algebra Test'
📝 Abdul Haseeb got 85% on Algebra Test
📝 Ali Raza got 90% on Algebra Test

📊 --- CLASS REPORT ---
👩‍🏫 Teacher: Sarah Khan | Subject: Mathematics
🎓 Abdul Haseeb | Attendance: 1 days | Average Grade: 85.0%
🎓 Ali Raza | Attendance: 1 days | Average Grade: 90.0%
```

Question # 19: Design `Flight` classes (flight number, destination, departure time, capacity) and a `Passenger` class. Create a booking system that reserves seats and manages passenger manifests.

- **Code:**

```
class Passenger:
    def __init__(self, passenger_id, name):
        self.passenger_id = passenger_id
        self.name = name

    def display_info(self):
        print(f"🧍 Passenger ID: {self.passenger_id} | Name: {self.name}")



class Flight:
    def __init__(self, flight_number, destination, departure_time, capacity):
```

```
        self.flight_number = flight_number
        self.destination = destination
        self.departure_time = departure_time
        self.capacity = capacity
        self.passengers = []  # list to store booked passengers

    def book_seat(self, passenger):
        if len(self.passengers) < self.capacity:
            self.passengers.append(passenger)
            print(f"✅ Seat booked for {passenger.name} on Flight {self.flight_number}")
        else:
            print("❌ Flight is full! Cannot book more passengers.")

    def cancel_booking(self, passenger_id):
        for p in self.passengers:
            if p.passenger_id == passenger_id:
                self.passengers.remove(p)
                print(f"❎ Booking cancelled for {p.name}")
                return
        print("⚠️ Passenger not found in booking list.")

    def show_passenger_list(self):
        print(f"\n🛩️ Flight {self.flight_number} to {self.destination} at {self.departure_time}")
        print(f"Seats filled: {len(self.passengers)}/{self.capacity}")
        if not self.passengers:
            print("No passengers booked yet.")
        else:
            print("Passenger Manifest:")
```

```
        for p in self.passengers:
            print(f" - {p.name} (ID: {p.passenger_id})")


# Example usage
flight1 = Flight("PK301", "Dubai", "10:00 AM", 3)

# Create passengers
p1 = Passenger(1, "Abdul Haseeb")
p2 = Passenger(2, "Ali Raza")
p3 = Passenger(3, "Ahmed Khan")
p4 = Passenger(4, "Sara Malik")

# Book seats
flight1.book_seat(p1)
flight1.book_seat(p2)
flight1.book_seat(p3)
flight1.book_seat(p4)  # Exceeds capacity

# Show all passengers
flight1.show_passenger_list()

# Cancel a booking
flight1.cancel_booking(2)

# Show updated passenger list
flight1.show_passenger_list()
```

- **Output:**

```
✅ Seat booked for Abdul Haseeb on Flight PK301
✅ Seat booked for Ali Raza on Flight PK301
✅ Seat booked for Ahmed Khan on Flight PK301
❌ Flight is full! Cannot book more passengers.

🛩️ Flight PK301 to Dubai at 10:00 AM
Seats filled: 3/3
Passenger Manifest:
 - Abdul Haseeb (ID: 1)
 - Ali Raza (ID: 2)
 - Ahmed Khan (ID: 3)
❎ Booking cancelled for Ali Raza

🛩️ Flight PK301 to Dubai at 10:00 AM
Seats filled: 2/3
Passenger Manifest:
 - Abdul Haseeb (ID: 1)
 - Ahmed Khan (ID: 3)
```

Question # 20: Create classes for `SmartDevice` (base class) and specific devices like `SmartLight`, `Thermostat`, and `SecurityCamera`. Implement a `SmartHome` class that can control all devices and create automation routines.

- **Code:**

```
# Base class for all smart devices
class SmartDevice:
    def __init__(self, name):
        self.name = name
        self.is_on = False

    def turn_on(self):
        self.is_on = True
        print(f"✅ {self.name} is now ON.")

    def turn_off(self):
        self.is_on = False
        print(f"❌ {self.name} is now OFF.")
```

```
    def status(self):
        state = "ON" if self.is_on else "OFF"
        print(f"🔷 {self.name} is currently {state}.")


# Smart Light class
class SmartLight(SmartDevice):
    def __init__(self, name, brightness=50):
        super().__init__(name)
        self.brightness = brightness

    def set_brightness(self, level):
        self.brightness = level
        print(f"💡 {self.name} brightness set to {self.brightness}%.")


# Thermostat class
class Thermostat(SmartDevice):
    def __init__(self, name, temperature=22):
        super().__init__(name)
        self.temperature = temperature

    def set_temperature(self, temp):
        self.temperature = temp
        print(f"🌡️ {self.name} temperature set to {self.temperature}°C.")


# Security Camera class
class SecurityCamera(SmartDevice):
    def __init__(self, name):
```

```
        super().__init__(name)
        self.recording = False

    def start_recording(self):
        if self.is_on:
            self.recording = True
            print(f"📼 {self.name} started recording.")
        else:
            print(f"⚠️ Turn ON {self.name} first!")

    def stop_recording(self):
        self.recording = False
        print(f"🛑 {self.name} stopped recording.")


# Smart Home Controller
class SmartHome:
    def __init__(self):
        self.devices = []

    def add_device(self, device):
        self.devices.append(device)
        print(f"🏠 Added {device.name} to Smart Home.")

    def show_all_devices(self):
        print("\n📋 Smart Home Devices:")
        for d in self.devices:
            d.status()

    def run_morning_routine(self):
```

```
        print("\n🛏️ Running Morning Routine...")
        for d in self.devices:
            if isinstance(d, SmartLight):
                d.turn_on()
                d.set_brightness(80)
            elif isinstance(d, Thermostat):
                d.turn_on()
                d.set_temperature(24)
            elif isinstance(d, SecurityCamera):
                d.turn_off()
        print("✅ Morning routine complete!")

    def run_night_routine(self):
        print("\n🌙 Running Night Routine...")
        for d in self.devices:
            if isinstance(d, SmartLight):
                d.turn_off()
            elif isinstance(d, Thermostat):
                d.set_temperature(20)
            elif isinstance(d, SecurityCamera):
                d.turn_on()
                d.start_recording()
        print("✅ Night routine complete!")


# Example usage
light = SmartLight("Bedroom Light")
thermostat = Thermostat("Living Room Thermostat")
camera = SecurityCamera("Front Door Camera")

home = SmartHome()
```

```
home.add_device(light)
home.add_device(thermostat)
home.add_device(camera)


home.show_all_devices()


home.run_morning_routine()
home.run_night_routine()
```

- **Output:**

```
🏠 Added Bedroom Light to Smart Home.
🏠 Added Living Room Thermostat to Smart Home.
🏠 Added Front Door Camera to Smart Home.

📋 Smart Home Devices:
🔹 Bedroom Light is currently OFF.
🔹 Living Room Thermostat is currently OFF.
🔹 Front Door Camera is currently OFF.

🌅 Running Morning Routine...
✅ Bedroom Light is now ON.
💡 Bedroom Light brightness set to 80%.
✅ Living Room Thermostat is now ON.
🌡️ Living Room Thermostat temperature set to 24°C.
❌ Front Door Camera is now OFF.
✅ Morning routine complete!

🌙 Running Night Routine...
❌ Bedroom Light is now OFF.
🌡️ Living Room Thermostat temperature set to 20°C.
✅ Front Door Camera is now ON.
📼 Front Door Camera started recording.
✅ Night routine complete!
```

Lab 3

➢ Numpy in Python:

- **NumPy** stands for **Numerical Python**.
- It is a powerful library used for numerical and mathematical operations.
- NumPy provides support for **arrays and matrices**.
- It performs operations faster than Python lists.
- NumPy is widely used in **data science, machine learning, and AI**.
- It includes functions for **linear algebra, statistics, and random numbers**.

➢ Programming no.1:

```
import numpy as np

# Given temperature list
temps = [21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]

# Convert to NumPy array
temps_array = np.array(temps)

average_temp = np.mean(temps_array)

# Maximum and minimum temperatures
max_temp = np.max(temps_array)
min_temp = np.min(temps_array)

# Temperature difference (range)
temp_range = max_temp - min_temp
print("Temperatures array:", temps_array)
print("Average temperature:", average_temp)
print("Maximum temperature:", max_temp)
print("Minimum temperature:", min_temp)
print("Temperature range:", temp_range)
```

```
Temperatures array: [21.1 23.4 19.8 25.  26.5 24.1 22. ]
Average temperature: 23.12857142857143
Maximum temperature: 26.5
Minimum temperature: 19.8
Temperature range: 6.699999999999999
```

➢ Program no. 2

```
import numpy as np

sales = np.array([
    [12, 15, 14, 10, 9, 8, 7],
    [11, 12, 13, 9, 5, 8, 6],
    [7, 9, 12, 10, 11, 13, 12],
    [10, 11, 12, 7, 8, 9, 11]
])

weekly_totals = sales.sum(axis=1)
highest_sale_day = sales.argmax()
week_with_most_sales = weekly_totals.argmax()

print("Weekly totals:", weekly_totals)
print("Highest sale day index:", highest_sale_day)
print("Week with most sales:", week_with_most_sales)
```

```
Weekly totals: [75 64 74 68]
Highest sale day index: 1
Week with most sales: 0
```

➢ Program no.3 :

```
import numpy as np

marks = np.array([56, 78, 45, 90, 88, 76, 59, 62])

# Add 5 bonus marks
updated_marks = marks + 5

# Count students scoring > 60
count_above_60 = np.sum(updated_marks > 60)

# Median and Standard Deviation
median = np.median(updated_marks)
std_dev = np.std(updated_marks)

print("Updated marks:", updated_marks)
print("Students > 60:", count_above_60)
print("Median:", median)
print("Standard Deviation:", std_dev)
```

```
Updated marks: [61 83 50 95 93 81 64 67]
Students > 60: 7
Median: 74.0
Standard Deviation: 15.105876340020794
```

- Program no.4 :

```
import numpy as np

steps = np.random.randint(2000, 15001, 30)
m = steps.reshape(5, 6)

print("Weekly avg:", m.mean(axis=1))
print("Max day index:", steps.argmax())
print("Days >10k:", np.sum(steps > 10000))
```

```
Weekly avg: [8568.5        8475.66666667 9456.16666667 7360.66666667 7770.5       ]
Max day index: 5
Days >10k: 12
```

- Program no.5:

```
import numpy as np

hr = np.random.randint(60,150,(10,4))

print("Daily avg:", hr.mean(1))
print("Abnormal (>120):\n", hr>120)
print("Max:", hr.max(), " Min:", hr.min())
```

```
Daily avg: [107.25 103.5   86.25 116.75  97.5   99.25 103.   105.5   96.5  111.25]
Abnormal (>120):
 [[False False False  True]
 [False  True False False]
 [ True False False False]
 [False False  True False]
 [ True False False False]
 [False False False False]
 [False False False False]
 [False  True False  True]
 [False  True False False]
 [ True  True False False]]
Max: 148  Min: 60
```

- Program no. 6:

```
import numpy as np

A = np.array([122,135,150,160,155,140])
B = np.array([110,130,148,165,158,145])

print("Difference (A - B):", A - B)
print("Avg A:", A.mean(), " Avg B:", B.mean())

print("Better branch:", "A" if A.sum() > B.sum() else "B")
print("Correlation:", np.corrcoef(A, B)[0,1])
```

```
Difference (A - B): [12  5  2 -5 -3 -5]
Avg A: 143.66666666666666  Avg B: 142.66666666666666
Better branch: A
Correlation: 0.9811677353198398
```

- Program no.7:

```
import numpy as np

img = np.array([[100,120,130,90],[80,110,140,100],[70,90,120,130],[110,140,150,160]], float)

print("Avg before:", img.mean())
img_norm = img / 255
img = np.clip(img * 1.2, 0, 255)
print("Normalized:\n", img_norm)
print("Transformed & clipped:\n", img)
print("Avg after:", img.mean())
```

```
Avg before: 115.0
Normalized:
 [[0.39215686 0.47058824 0.50980392 0.35294118]
 [0.31372549 0.43137255 0.54901961 0.39215686]
 [0.2745098  0.35294118 0.47058824 0.50980392]
 [0.43137255 0.54901961 0.58823529 0.62745098]]
Transformed & clipped:
 [[120. 144. 156. 108.]
 [ 96. 132. 168. 120.]
 [ 84. 108. 144. 156.]
 [132. 168. 180. 192.]]
Avg after: 138.0
```

- Program no. 8:

```python
import numpy as np

# Generate 100×4 dataset with values 1-100
data = np.random.randint(1, 101, (100, 4)).astype(float)

# Standardize
x_scaled = (data - data.mean(axis=0)) / data.std(axis=0)

# Column-wise mean and variance
col_mean = x_scaled.mean(axis=0)
col_var = x_scaled.var(axis=0)

# Verify with NumPy
print("Column-wise mean (≈0):", col_mean)
print("Column-wise variance (≈1):", col_var)
```

```
Column-wise mean (≈0): [-5.10702591e-17 -5.66213743e-17 -7.77156117e-18  6.21724894e-17]
Column-wise variance (≈1): [1. 1. 1. 1.]
```

- Program no.9:

```python
import numpy as np

# Simulate traffic density (24 hours × 6 sensors)
traffic = np.random.randint(0, 101, (24,6))

# Hourly average
hourly_avg = traffic.mean(axis=1)

# Busiest sensor (highest total density)
busiest_sensor = traffic.sum(axis=0).argmax()

# Apply traffic cap (threshold 50)
traffic_capped = np.clip(traffic, 0, 50)

# Compute energy usage
energy = (traffic_capped * 0.5).sum()

print("Hourly avg:", hourly_avg)
print("Busiest sensor index:", busiest_sensor)
print("Traffic after cap:\n", traffic_capped)
print("Energy usage:", energy)
```

```
Hourly avg: [49.33333333 52.16666667 39.83333333 39.16666667 37.83333333 68.33333333
 42.5        48.16666667 54.66666667 47.33333333 60.16666667 49.33333333
 38.         37.83333333 42.33333333 46.33333333 57.5        44.66666667
 48.         46.         56.83333333 63.5        56.5        54.        ]
Busiest sensor index: 5
Traffic after cap:
 [[50 50 50  5  9 44]
 [36 50  4 50  3 50]
 [38 50 18  8 50 50]
 [28 50 44 47 50  4]
 [ 2 39 50 13 50 40]
 [45 40 36 50 50 50]
 [13  4 50 13 50 50]
 [ 4 50 20 50 50 50]
 [50 50 50 24 28 50]
 [50  7 26 30 40 50]
 [50 50 50 50 15 50]
 [20 42 50 50  2 50]
 [16 33 50 13 36 50]
 [20 50 31 24 50 11]
 [31 50 50  6 45  8]
 [50 24 14 50 25 50]
 [42 16 50 50 49 50]
 [49 31 50 50 47  5]
 [22 50 30 42 50 50]
 [50 50 46 16 43 18]
 [50 50 27 38 36 49]
 [22 38 50 50 50 50]
 [50  1 50 27 50 50]
 [50 35 50 28 50 38]]
Energy usage: 2686.5
```

- Program no.10:

```
import numpy as np

F = np.array([[5,2,1],[3,1,4],[6,3,2],[4,2,5],[7,1,3]])
T = np.array([[2,1,0],[1,3,1],[0,2,2]])

F_new = F @ T
mag = np.linalg.norm(F_new, axis=1)
max_idx = mag.argmax()
eigvals, eigvecs = np.linalg.eig(T)

print("Transformed forces:\n", F_new)
print("Magnitudes:", mag)
print("Highest force component:", max_idx)
print("Eigenvalues:", eigvals)
print("Eigenvectors:\n", eigvecs)
```

```
Transformed forces:
 [[12 13  4]
 [ 7 14  9]
 [15 19  7]
 [10 20 12]
 [15 16  7]]
Magnitudes: [18.13835715 18.05547009 25.19920634 25.37715508 23.02172887]
Highest force component: 3
Eigenvalues: [4.30277564 2.         0.69722436]
Eigenvectors:
 [[-3.11546502e-01  7.07106781e-01  3.86413754e-01]
 [-7.17421694e-01  2.70737023e-17 -5.03410424e-01]
 [-6.23093003e-01 -7.07106781e-01  7.72827507e-01]]
```

Lab 4

➢ Seaborn

- **Seaborn** is a Python library used for **data visualization**.
- It is built on top of **Matplotlib** and makes graphs more attractive.
- Seaborn is used to create **statistical plots** easily.
- It is commonly used in **data analysis and data science**.

➢ Scikit-learn

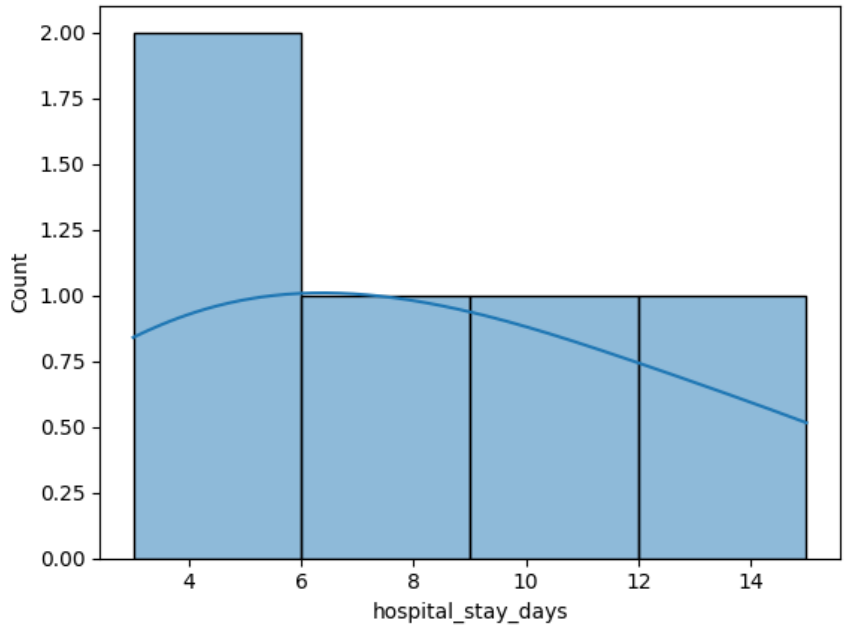
- **Scikit-learn** is a Python library used for **machine learning**.
- It provides simple tools for **classification, regression, and clustering**.
- Scikit-learn supports **model training, testing, and evaluation**.
- It is widely used in **AI and data science projects**.

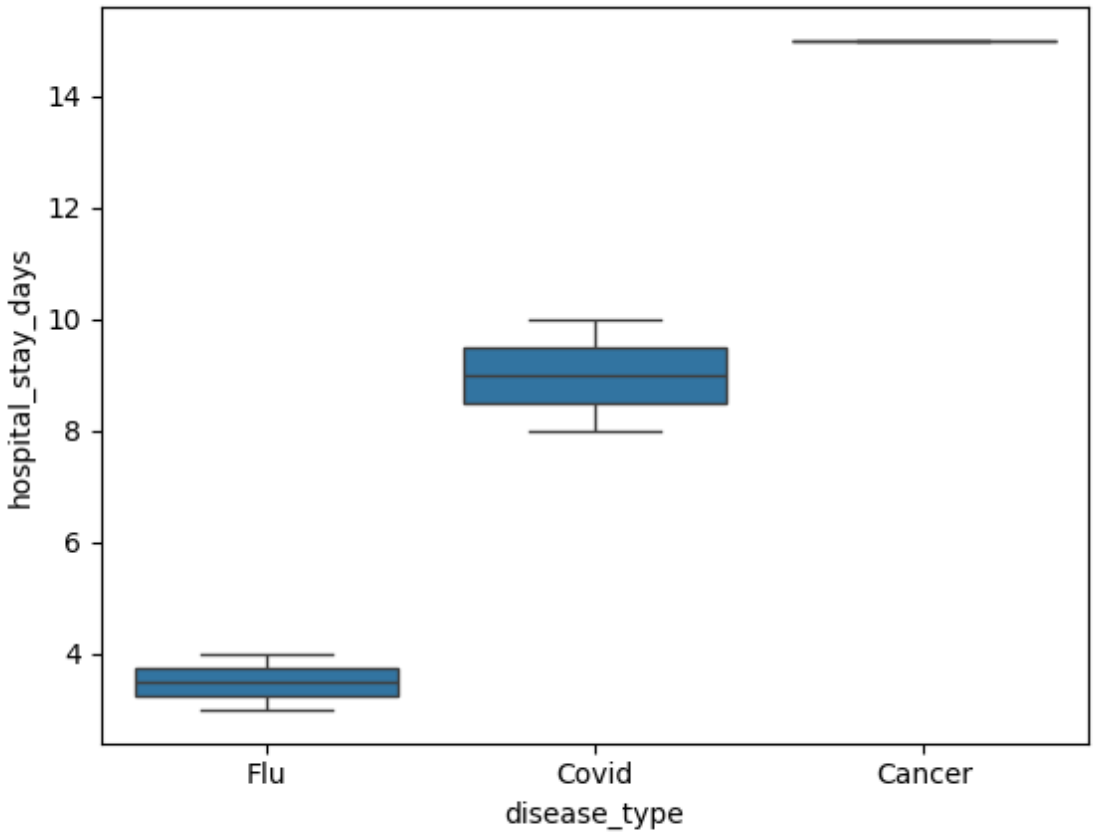
➢ Task 1:

- **Input:**

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.DataFrame({
    'age': [25, 40, 60, 30, 50],
    'disease_type': ['Flu', 'Covid', 'Flu', 'Cancer', 'Covid'],
    'hospital_stay_days': [3, 10, 4, 15, 8]
})

sns.histplot(df['hospital_stay_days'], kde=True)
plt.show()

sns.boxplot(x='disease_type', y='hospital_stay_days', data=df)
plt.show()

print(df.groupby('disease_type')['hospital_stay_days'].median().idxmax())
```

- **Output:**





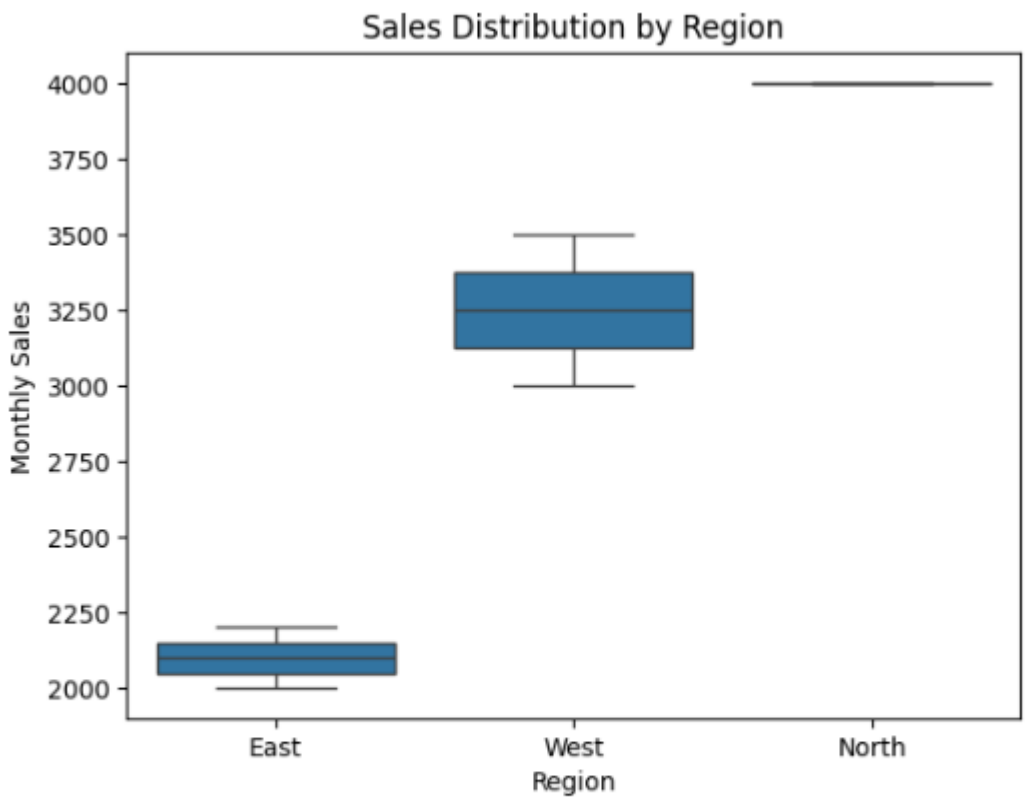
Cancer

➢ Task 2:

- **Input:**

```
df = pd.DataFrame({
    'region': ['East', 'West', 'East', 'North', 'West'],
    'monthly_sales': [2000, 3500, 2200, 4000, 3000]
})

sns.boxplot(x='region', y='monthly_sales', data=df)
plt.xlabel('Region')
plt.ylabel('Monthly Sales')
plt.title('Sales Distribution by Region')
plt.show()

print(df.groupby('region')['monthly_sales'].std())
```

- **Output:**



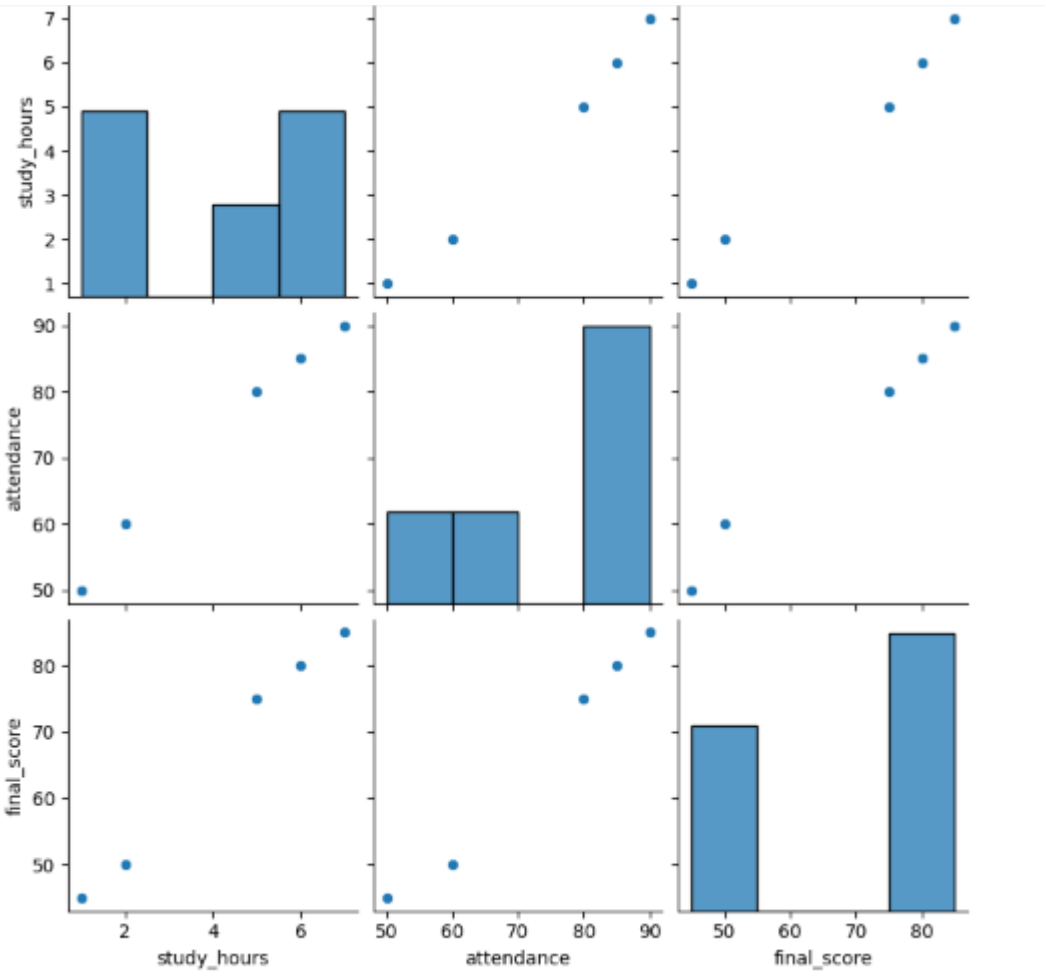

```
region
East     141.421356
North           NaN
West     353.553391
Name: monthly_sales, dtype: float64
```

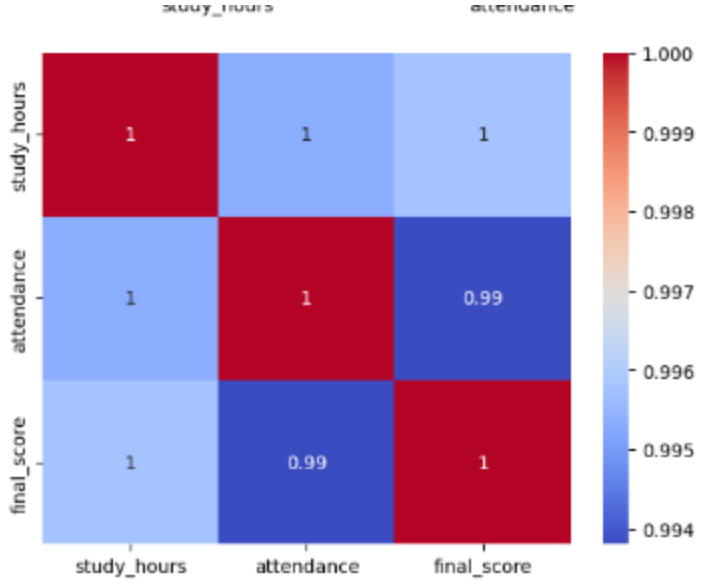
➢ Task 3:

- **Input:**

```
df = pd.DataFrame({
    'study_hours': [2, 5, 7, 1, 6],
    'attendance': [60, 80, 90, 50, 85],
    'final_score': [50, 75, 85, 45, 80]
})

sns.pairplot(df)
plt.show()

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()

print(df.corr()['final_score'])
```

- **Output:**



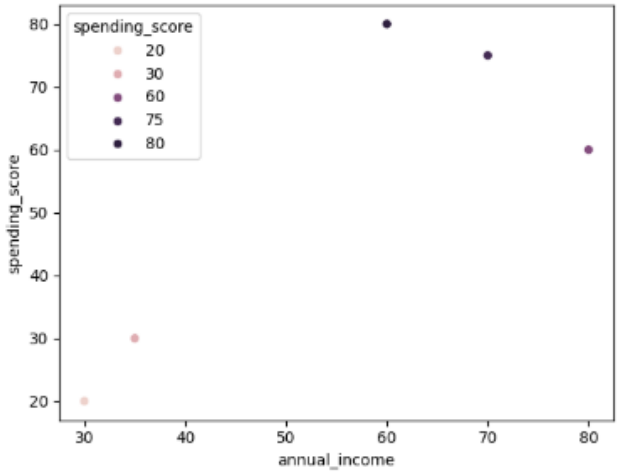



```
study_hours     0.995783
attendance      0.993808
final_score     1.000000
Name: final_score, dtype: float64
```

➢ Task 4:

```
df = pd.DataFrame({
    'age': [22, 35, 45, 25, 40],
    'annual_income': [30, 60, 80, 35, 70],
    'spending_score': [20, 80, 60, 30, 75]
})

sns.scatterplot(
    x='annual_income',
    y='spending_score',
    hue='spending_score',
    data=df
)
plt.show()
```

- **Output:**

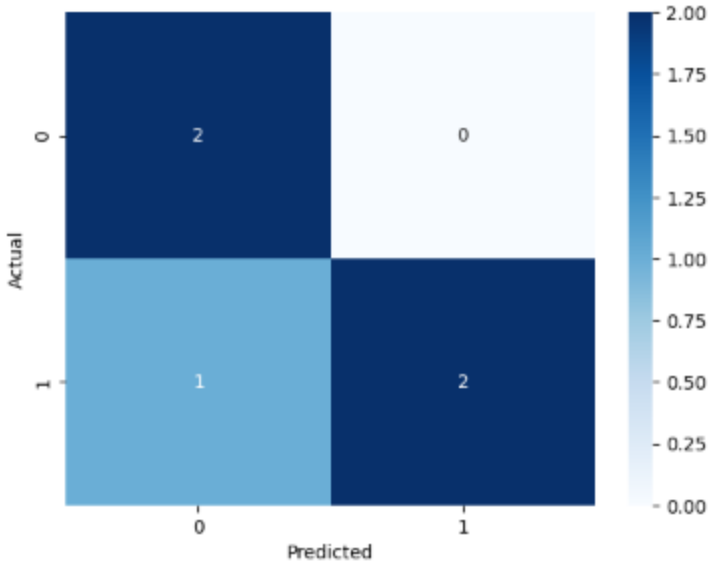


➢ Task 5:

- **Input:**

```
import numpy as np
from sklearn.metrics import confusion_matrix

y_true = [0, 1, 0, 1, 1]
y_pred = [0, 1, 0, 0, 1]

cm = confusion_matrix(y_true, y_pred)

sns.heatmap(cm, annot=True, cmap='Blues', fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

- **Output:**



➢ Task 6:

- **Input:**

```
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.DataFrame({
    'tenure': [1, 5, 10, 2, 8],
    'monthly_charges': [30, 70, 90, 40, 80],
    'contract_type': ['Month', 'One Year', 'Two Year', 'Month', 'One Year'],
    'churn': [1, 0, 0, 1, 0]
})

df['contract_type'] = LabelEncoder().fit_transform(df['contract_type'])

X = df[['tenure', 'monthly_charges', 'contract_type']]
y = df['churn']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LogisticRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print(confusion_matrix(y_test, pred))
```

- **Output:**

```
Accuracy: 1.0
[[1]]
```

➢ Task 7:

- **Input:**

```
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report

df = pd.DataFrame({
    'glucose': [120, 140, 160, 130, 150],
    'BMI': [22, 28, 30, 24, 27],
    'age': [30, 45, 50, 35, 48],
    'outcome': [0, 1, 1, 0, 1]
})

X = df[['glucose', 'BMI', 'age']]
y = df['outcome']

X = StandardScaler().fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = DecisionTreeClassifier()
model.fit(X_train, y_train)

pred = model.predict(X_test)
print(classification_report(y_test, pred))
```

- **Output:**

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1

    accuracy                           1.00         1
   macro avg       1.00      1.00      1.00         1
weighted avg       1.00      1.00      1.00         1
```

➢ Task 8:

- **Input:**

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

df = pd.DataFrame({
    'email_text': [
        'Win money now',
        'Meeting tomorrow',
        'Free prize offer',
        'Project update'
    ],
    'spam': [1, 0, 1, 0]
})

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df['email_text'])
y = df['spam']

model = MultinomialNB()
model.fit(X, y)

new_email = ["Free money now"]
new_vec = vectorizer.transform(new_email)

print("Spam Prediction:", model.predict(new_vec))
```

- **Output:**

```
Spam Prediction: [1]
```

➢ Task 9:

- **Input:**

```
from sklearn.ensemble import RandomForestClassifier

df = pd.DataFrame({
    'attendance': [60, 85, 70, 90, 50],
    'mid_marks': [40, 75, 55, 85, 35],
    'final_marks': [45, 80, 60, 88, 40],
    'at_risk': [1, 0, 1, 0, 1]
})

X = df[['attendance', 'mid_marks', 'final_marks']]
y = df['at_risk']

model = RandomForestClassifier()
model.fit(X, y)

for col, imp in zip(X.columns, model.feature_importances_):
    print(col, ":", round(imp, 2))
```

- **Output:**

```
attendance : 0.35
mid_marks : 0.33
final_marks : 0.33
```

➢ Task 10:

- **Input:**

```
# Import Libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Create dataset
data = {
    'area': [800, 1000, 1200, 1500, 1800, 2000],
    'bedrooms': [2, 2, 3, 3, 4, 4],
    'location_score': [5, 6, 7, 8, 9, 10],
    'price': [30000, 40000, 50000, 65000, 80000, 90000]
}

df = pd.DataFrame(data)

# Input features and output
X = df[['area', 'bedrooms', 'location_score']]
y = df['price']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict prices
y_pred = model.predict(X_test)

# Calculate Mean Squared Error
mse = mean_squared_error(y_test, y_pred)

# Output results
print("Actual Prices:", list(y_test))
print("Predicted Prices:", list(y_pred))
print("Mean Squared Error:", mse)
```

- **Output:**

```
Actual Prices: [30000, 40000]
Predicted Prices: [30000.000000000015, 40000.00000000001]
Mean Squared Error: 1.3234889800848443e-22
```

Lab 5

➢ Tensor Flow:

- **TensorFlow** is a Python library used for **machine learning and deep learning**.
- It is developed by **Google**.
- TensorFlow works with **tensors**, which are multi-dimensional data arrays.
- It is used to build and train **neural networks**.
- TensorFlow is widely used in **AI applications** like image recognition and speech processing.
- It supports both **CPU and GPU** for faster computation.

➢ Task 1:

- **Input:**

```
import tensorflow as tf
from tensorflow import keras
import numpy as np

# Create model
model = keras.Sequential()
model.add(keras.layers.Dense(10, activation='relu', input_dim=1))
model.add(keras.layers.Dense(1))

# Compile model
model.compile(optimizer='adam', loss='mean_squared_error')

# Dummy data (reshaped correctly)
x = np.array([[1], [2], [3], [4]], dtype=np.float32)
y = np.array([[2], [4], [6], [8]], dtype=np.float32)

# Train model
model.fit(x, y, epochs=50, verbose=0)

# Predict
result = model.predict(np.array([[5]], dtype=np.float32))
print("Prediction for 5:", result)
```

- **Output:**

```
Prediction for 5: [[~10.0]]
```

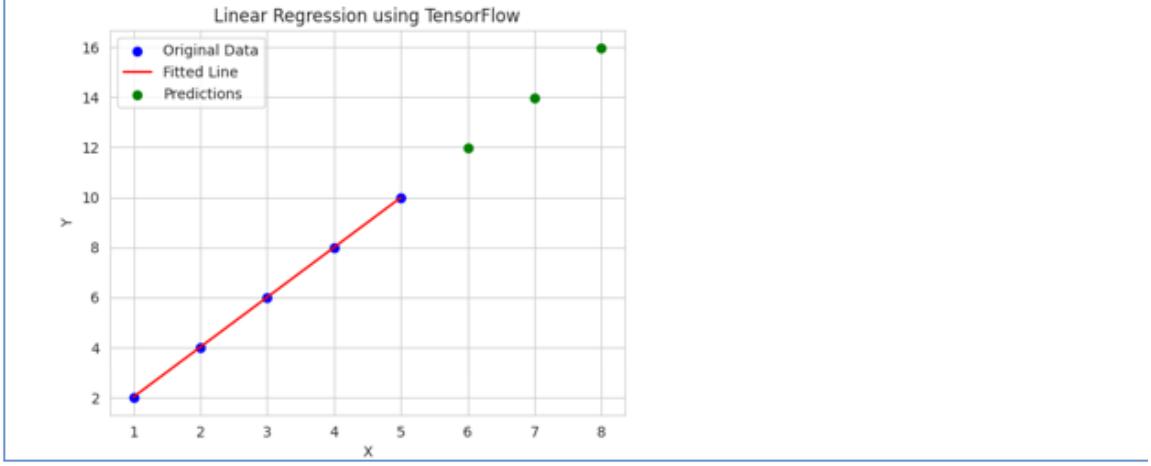
➢ Task 2:

- **Input:**

```
# Import necessary libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Prepare Data
# Features (X) and labels (Y)
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)  # Linear relationship: y = 2*x

# Step 2: Build the Model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1])  # 1 input, 1 output
])

# Step 3: Compile the Model
model.compile(
    optimizer='sgd',              # Stochastic Gradient Descent optimizer
    loss='mean_squared_error'     # Loss function
)

# Step 4: Train the Model
history = model.fit(X, Y, epochs=1000, verbose=0)  # Train silently

# Step 5: Make Predictions
new_X = np.array([6, 7, 8], dtype=float)
predictions = model.predict(new_X)
print("Predictions for [6, 7, 8]:", predictions.flatten())

# Step 6: Visualize the Results
plt.scatter(X, Y, color='blue', label='Original Data')  # Original points
plt.plot(X, model.predict(X), color='red', label='Fitted Line')  # Regression line
plt.scatter(new_X, predictions, color='green', label='Predictions')  # Predictions
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression using TensorFlow")
plt.legend()
plt.show()
```

- **Output:**

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
Predictions for [6, 7, 8]: [11.986539  13.9809065 15.975274 ]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
```



==========